



ZW3D API 简介

本文档仅提供 ZW3D API 及相关开发工具的基本介绍，您可以借助它们实现对 ZW3D 软件的二次开发，从而实现您的客制化及自动化需求。我们推荐使用 C/C++ 开发基于 ZW3D 的附加组件。我们将不断改进 API，使其更加支持我们合伙人基于 ZW3D 开发强大的应用程序。

- 在第一章中主要介绍一些 ZW3D 二次开发的相关机制以及注意事项。
- 在第二章中我们会演示给您如何快速地开始 ZW3D API 的使用，包括如何创建一个简单的项目，如何调试程序以及过程中的注意事项。
- 在第三章中我们会演示给您如何自定义菜单、ribbon 栏及工具栏，帮助您快速部署所需的功能按键，如果您是从老版本迁移功能的客户，我们还提供了快速升级配置文件的工具，帮助您快速完成 zcui 文件的更新。
- 在第四章中我们会演示给您如何利用开发工具自定义 UI，您可以借助 UI 编辑工具创建所需的 UI 界面，并且能够通过相关命令在 ZW3D 中唤起该界面。
- 在第五章中我们会演示给您如何编辑 Tcmd 文件。Tcmd 文件会在创建模板命令时伴随 UI 文件创建，您可以借助 Tcmd 编辑器对其进行进一步的配置。详细的开发帮助及指引可以查询文档《Tcmd Editor Introduction.pdf》。
- 第六章为 ZW3D 二次开发中常见问题解答。
- 第七章为 ZW3D 特殊符号转义字符映射表。

如您有任何建议或诉求，请随时联系我们。

您可以邮件到 zdncentre@zwsoft.com 寻求帮助。

谢谢合作！

中望®、中望软件®、中望 3D®、中望 CAD®、ZW3D®、ZWCAD®、ZWSOFT®、overdrive® 及其图案均为广州中望龙腾软件股份有限公司已合法注册的商标。

打印于中华人民共和国。



目录

第一章 认识 ZW3D API	3
第二章 创建 ZW3D API 工程	6
第三章 自定义菜单.....	17
第四章 ZW3D UI 开发	28
第五章 ZW3D Tcmd 开发	41
第六章 常见问题解答.....	54
第七章 ZW3D 特殊符号转义字符映射表.....	57



第一章 认识 ZW3D API

1. ZW3D API 介绍

ZW3D API 提供了基于 C 语言的开发接口，功能涵盖实体创建、属性管理、信息查询、零件装配、工程出图等诸多方面，可以完成基于平台的功能拓展开发。详细的接口列表及其功能您可以通过查询接口配套的帮助文档"apiHelpDoc.chm"来进行开发和学习。

2. 新旧接口转换

为了更好地上手 ZW3D 的二次开发，在此我们对 ZW3D 二次开发的相关机制进行简单的介绍来帮助您快速了解。

由于 ZW3D 平台功能的持续性升级，且旧接口的历史跨度较大，一些接口的功能和适配范围存在不足，不能很好地满足您的使用需求。因此在 2025 及之后版本中我们对旧版接口进行了全面性的升级，为旧接口提供对应的功能更强大的新接口。同时为了最大限度维持老客户已有插件的功能，我们会在至少三个版本中对旧接口进行保留。但是由于平台功能存在的机制性的差异，一些旧接口的稳定性和功能范围与新接口存在一定差距，因此我们建议您逐步使用新接口替换旧接口。我们为旧接口提供了弃置声明和对应新接口的链接，您可以快速地查询到接口的升级信息。

旧版和新版两种接口分别以"cvx"和"Zw"开头，您可以通过接口名称快速地知道它是新接口还是旧接口。其中以"cvx"开头的是旧接口，旧接口通过实体编号"id"和实体路径"Path"来标识一个文件中的实体对象，如组件、面等。而新接口以"Zw"开头，它使用句柄"Handle"的方式进行标识，并提供了新旧接口标志的转换接口：

旧实体标记到新实体标记，通过实体 id 获取实体的句柄，

```
ezwErrors ZwEntityIdTransfer (int count, const int *indexes, szwEntityHandle *entityHandles);
```

旧实体标记到新实体标记，通过实体路径获取实体的句柄，

```
ezwErrors ZwEntityPathTransfer (int count, const svxEntPath *entityPaths, szwEntityHandle *entityHandles);
```

新实体标记到旧实体标记，通过实体的句柄获取实体 id，

```
ezwErrors ZwEntityIdGet (int count, const szwEntityHandle *entityHandles, int *indexes);
```

新实体标记到旧实体标记，通过实体的句柄获取实体路径，

```
ezwErrors ZwEntityPathGet (int count, const szwEntityHandle *entityHandles, svxEntPath *entityPaths);
```

其中"Entity"是我们的通用类对象，是点、线、面、体、特征等常规建模对象的合集，使用句柄的方式相比旧接口的对象标识方式对象更加唯一，您可以更直接地对对象进行查询类操作。需要注意的是句柄的标识方式会在 ZW3D 中申请一定的内存，因此您在使用接口完成后需要进行内存的释放。

3. 数据初始化

在您使用接口前，请您对使用到的变量用 C 或 C++的方法进行初始化。对于一些结



构复杂的接口，为了方便您对参数的设置，我们提供了配套的初始化函数。初始化函数以"Init"结尾，您可以通过接口名称或注释来快速查询和使用初始化函数。由于一些参数的默认值并不为零，因此建议您在接口使用时先进行初始化操作。以下是拉伸特征接口和它的初始化函数：

```
ezwErrors ZwFeatureExtrudeInit(szwExtrudeData *extrude);
ezwErrors ZwFeatureExtrudeCreate(szwExtrudeData extrude, szwEntityHandle
*shapeHandle);
```

一般情况下接口的入参是与平台功能设置项对应的，借助初始化函数填充与平台初始值一致的默认值能有效地提高接口的使用效率，并且能更好地帮助您确定接口参数的实际含义。当您不确定接口的某个参数时，可以通过相关注释和平台原始功能的对照进行确定。

4. 内存管理

由于 ZW3D 的部分接口会为您传入的参数分配内存，在使用完成后需要使用对应的接口来释放内存，避免出现内存问题。我们对于需要释放内存的接口进行了注释，您可以采用注释中提及的释放接口或通用接口可以完成对象内存的管理。以下是几种使用的方式示例：

- a) 使用单个 szwEntityHandle 对象：

```
szwEntityHandle handle;
ZwActiveLCSTGet(&handle);
ZwEntityHandleFree(&handle);
```

在上述代码中，接口"ZwActiveLCSTGet"会为变量 handle 分配内存，当您不再需要使用变量 handle 时，请调用接口"ZwEntityHandleFree"释放已分配的内存。

- b) 已知长度的 szwEntityHandle 数组：

```
假设已存在一个确定长度的 count 变量及对应的实体 id 数组 idList,
szwEntityHandle handles[count];
ZwEntityIdTransfer(count, idList, handles);
for (int i = 0; i < count; ++i)
{
    ZwEntityHandleFree(&handles[i]);
}
```

如果您使用 new 或者 alloc 动态地分配内存，请您记得释放相关内存。在上述代码中，需要为数组 handles 的每一位成员释放内存。

- c) 未知长度的 szwEntityHandle 数组：

```
int entityCount = 0;
szwEntityHandle *handleList = NULL; // nullptr for C++
ZwCurveListGet(&entityCount, &handleList);
ZwEntityHandleListFree(entityCount, &handleList);
```

在上述代码中，接口"ZwCurveListGet"会为变量 handleList 分配内存，并输出该数组的长度，当您不再需要使用变量 handleList 时，请调用接口"ZwEntityHandleListFree"释放已分配的内存。



d) 接口存在特定释放接口:

```
szwEntityHandle curvehandle;  
int nurbs;  
szwCurve curve;  
ZwCurveNURBSDataGet(curvehandle, nurbs, &curve);  
ZwEntityHandleFree(&curvehandle);  
ZwCurveNURBSDataFree(&curve);
```

在上述代码中, 接口"ZwCurveNURBSDataGet"为 curve 分配了内存, 在使用完成后需要使用特定的接口"ZwCurveNURBSDataFree"来释放内存, 另外使用到的其他句柄或占用内存的对象不要忘记释放内存。

e) 自主分配内存时:

当您使用"ZwMemoryAlloc"或其他方式申请了内存后, 同样需要在使用完成后对其进行释放。使用"Zw"内部的内存分配方式可以用"ZwMemoryFree"的方式释放内存, 当您使用自定义的方式管理内存时请自主释放您的内存。

5. 接口返回值

ZW3D 2025 之后新增的接口均使用"ezwErrors"作为返回值, 您需要判断每次调用接口后的返回值以确定该接口是否正常执行, 接口执行成功时将返回"ZW_API_NO_ERROR", 其他值为执行失败的结果。如果接口执行失败, 接口的输出参数将不会如您的预期, 此时请不要使用输出的结果。



第二章 创建 ZW3D API 工程

1. 系统要求

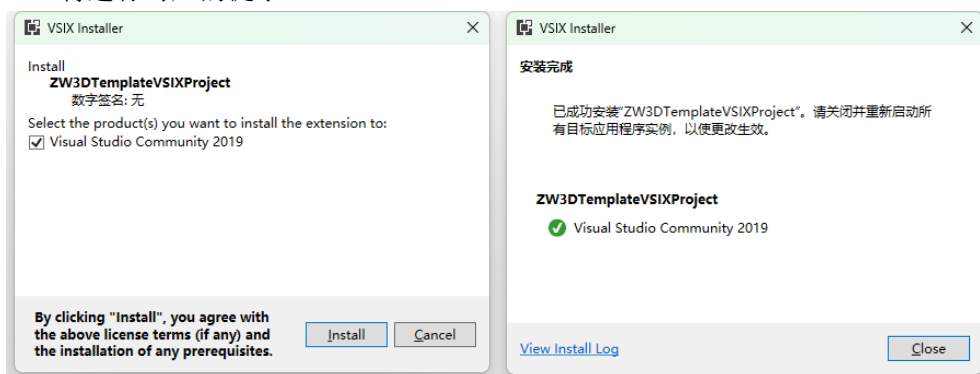
- a) Windows 7 SP1 或 Windows 10 或 Windows 11（推荐 Windows 10）
- b) Visual studio 2019（或 C/C++的任何其他集成开发环境）
- c) ZW3D 2023 或以上版本（建议使用与 ZW3D 一致的 API 的版本）
- d) Qt5.9.7(for Windows)

注意：您在安装 Qt 之后，参考第四章的方式安装 UI 编辑插件，即可使用 ZW3D 提供的自定义控件来完成 ZW3D 平台上的 UI 设计。

2. 安装二次开发向导模板

我们提供了二次开发向导模板让您可以快速地开启 ZW3D 二次开发之旅，省去繁杂的环境配置工作。在 api 文件夹内找到名为"ZW3DTemplateVSIXProject.vsix"的 Visual Studio 插件安装程序。双击该安装程序会自动检测本地的 Visual Studio 平台（适配 2017、2019、2022 版本），双击后根据提示安装向导模板，这个过程可能花费数分钟时间。

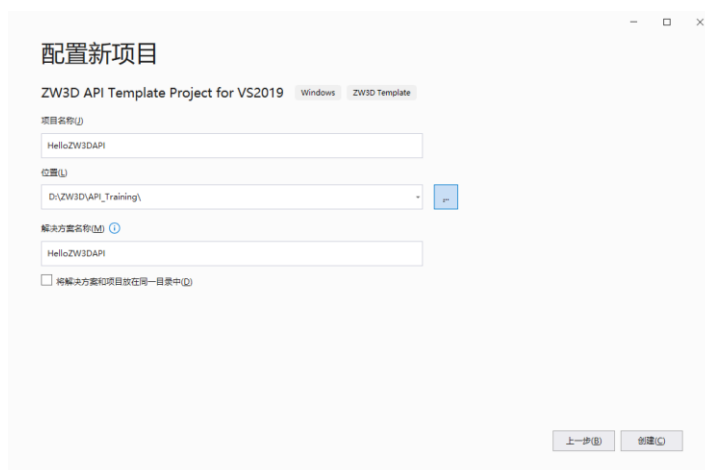
- a) 勾选您想要安装的版本，点击安装将自动安装对应二次开发向导模板，安装成功后将进行对应的提示。



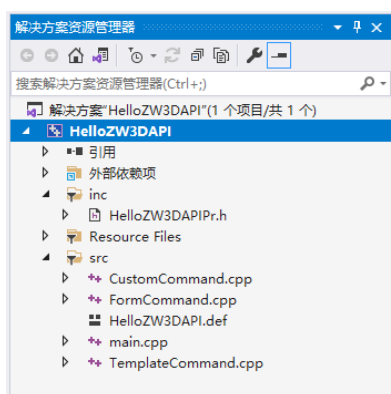
- b) 根据您的 Visual Studio 版本选择对应的模板直接创建开发工程。



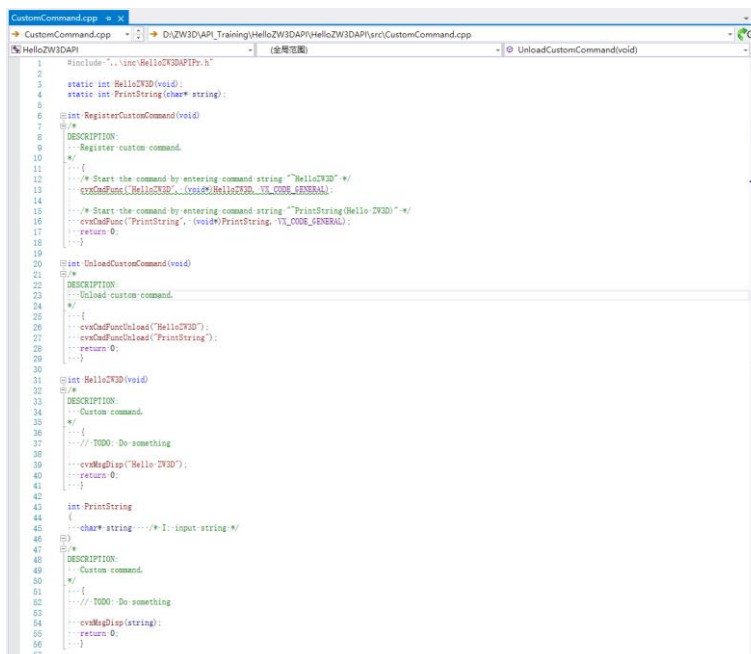
c) 创建名为"HelloZW3DAP1"的开发工程。



d) 模板中包含可供编辑的项目模板和测试案例。

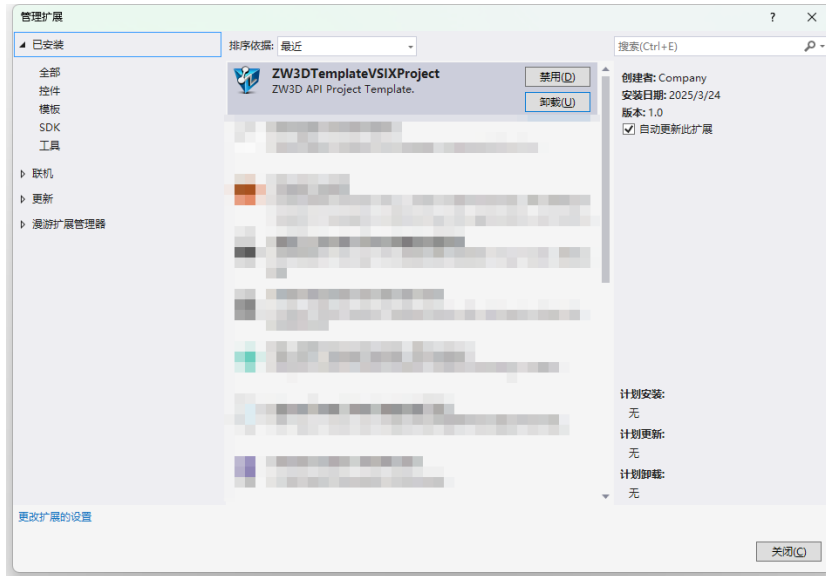


e) 打开"CustomCommand.cpp"可以看到提供的测试案例"HelloZW3DAPI"。





- f) 当您需要卸载开发工具时，您可以通过 Visual Studio 的插件管理找到对应的模板插件"ZW3DTemplateVSIXProject"，点击卸载后按照指引即可卸载本开发模板。

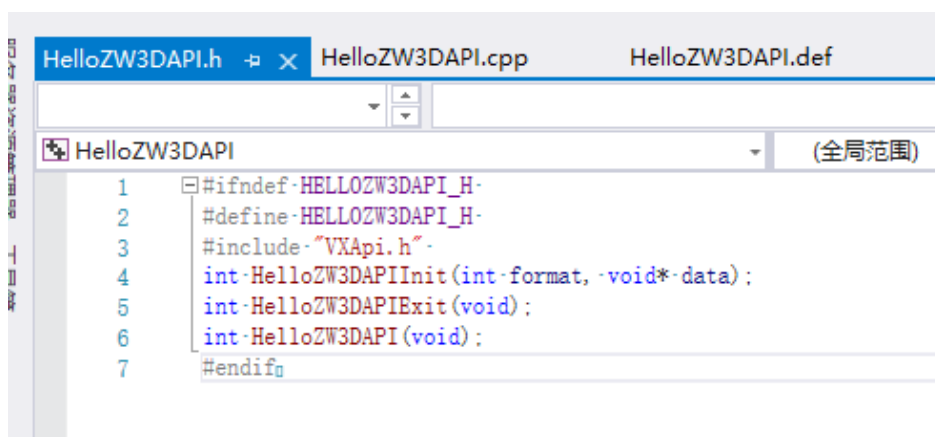


3. 配置环境

当您电脑未安装 ZW3D 程序、不使用向导模板及一些其他原因导致工程环境存在问题时，您可以按照下文的指示创建文件并进行环境的配置。

- a) 参考二次开发向导模板的内容创建"HelloZW3DAPI.h"。

```
#ifndef HELLOZW3DAPI_H
#define HELLOZW3DAPI_H
#include "VXApi.h"
int HelloZW3DAPIInit(int format, void* data);
int HelloZW3DAPIExit(void);
int HelloZW3DAPI(void);
#endif
```



- b) 参考二次开发向导模板的内容创建"HelloZW3DAPI.cpp"。

```
#include "HelloZW3DAPI.h"
int HelloZW3DAPIInit(int format, void* data)
```



```
{
    cvxCmdFunc("HelloZW3DAPI", (void*)HelloZW3DAPI, VX_CODE_GENERAL);
    return 0;
}
int HelloZW3DAPIExit(void)
{
    cvxCmdFuncUnload("HelloZW3DAPI");
    return 0;
}
int HelloZW3DAPI(void)
{
    cvxMsgDisp("Hello ZW3D API!");
    return 0;
}
```

```
HelloZW3DAPI.h  HelloZW3DAPI.cpp  HelloZW3DAPI.def
HelloZW3DAPI.cpp  D:\ZW3D\API_Training\HelloZW3DAPI\HelloZW3DAPI\HelloZW3DAPI.cpp
HelloZW3DAPI  (全局范围)
1  #include "HelloZW3DAPI.h"
2  int HelloZW3DAPIInit(int format, void* data)
3  {
4      cvxCmdFunc("HelloZW3DAPI", (void*)HelloZW3DAPI, VX_CODE_GENERAL);
5      return 0;
6  }
7  int HelloZW3DAPIExit(void)
8  {
9      cvxCmdFuncUnload("HelloZW3DAPI");
10     return 0;
11 }
12 int HelloZW3DAPI(void)
13 {
14     cvxMsgDisp("Hello ZW3D API!");
15     return 0;
16 }
```

c) 参考二次开发向导模板的内容创建"HelloZW3DAPI.def"。

LIBRARY HelloZW3DAPI.dll

EXPORTS

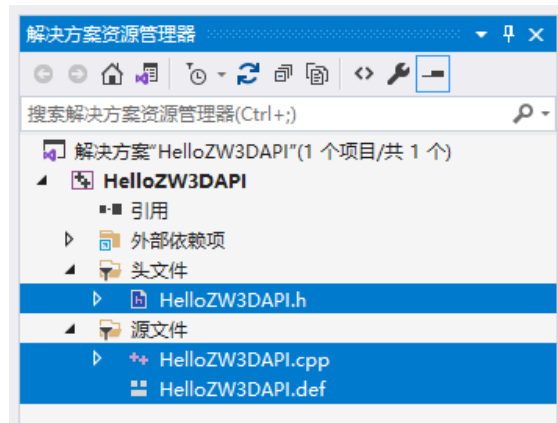
HelloZW3DAPIInit

HelloZW3DAPIExit

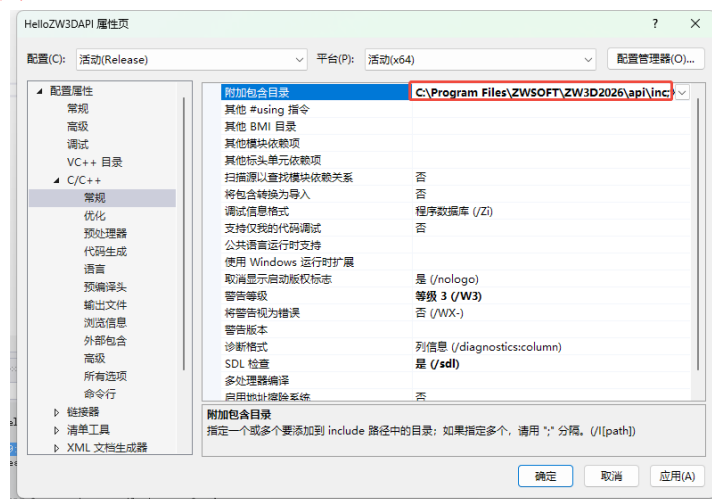
HelloZW3DAPI

```
HelloZW3DAPI.h  HelloZW3DAPI.cpp  HelloZW3DAPI.def
1  LIBRARY HelloZW3DAPI.dll
2  EXPORTS
3  HelloZW3DAPIInit
4  HelloZW3DAPIExit
5  HelloZW3DAPI
```

d) 将"HelloZW3DAPI.h"、"HelloZW3DAPI.cpp"和"HelloZW3DAPI.def"添加到项目中。

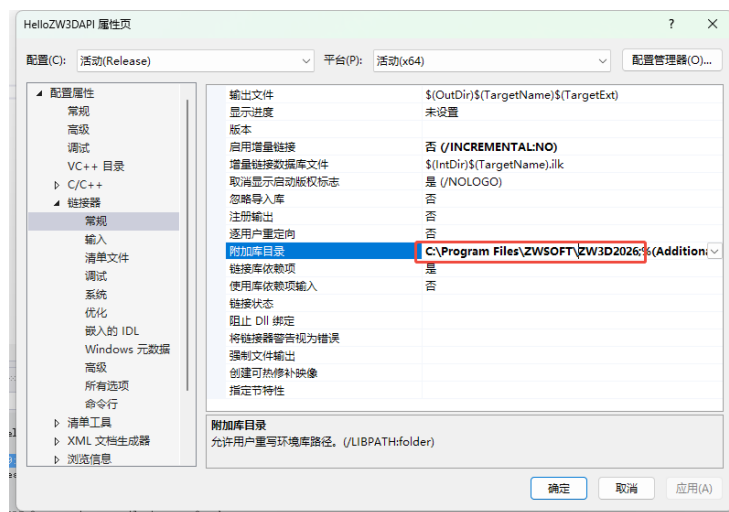


e) 右键点击项目，选择属性，将 ZW3D API 头文件目录添加至：**C/C++ → 常规 → 附加包含目录**。

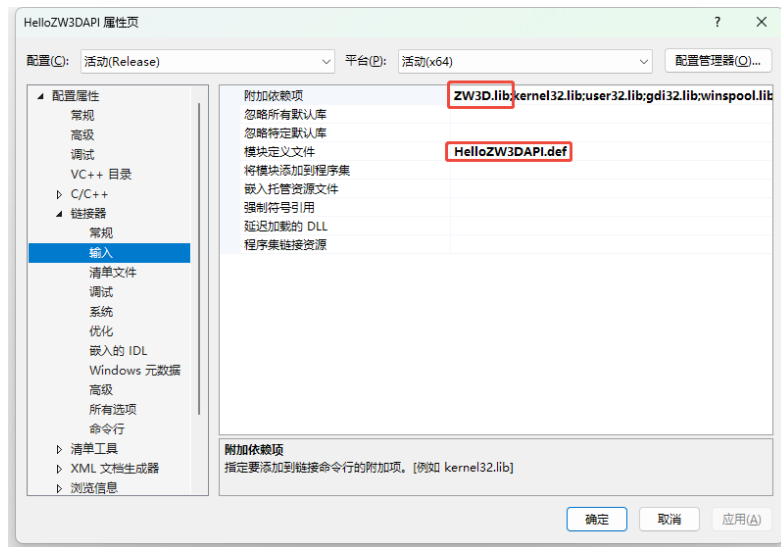


注意：如果您的版本早于 ZW3D 2024，包含目录应该移除"inc"。

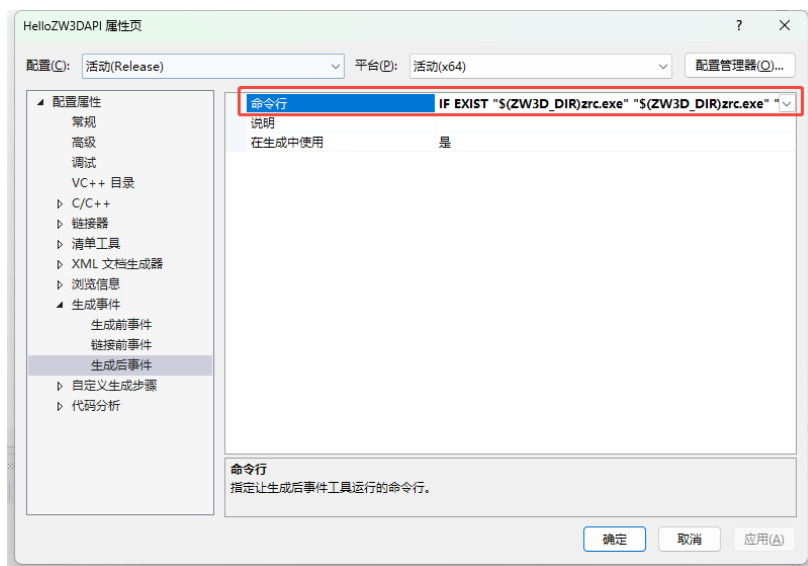
f) 将 ZW3D API 库文件目录添加至：**链接器 → 常规 → 附加库目录**。



- g) 将 ZW3D API 库添加至：**链接器→输入→附加依赖项**。def 文件添加至：**链接器→输入→模块定义文件**。



- h) 要编译资源，请将以下的命令行添加至：**生成事件→后期生成事件→命令行**。



**IF EXIST "\$(ZW3D_DIR)zrc.exe" "\$(ZW3D_DIR)zrc.exe" "\$(SolutionDir)." -o
"\$(TargetDir)\$\$(ProjectName).zrc"**

注意：上述命令为连续的一行内容，请勿将其当作两行内容。无资源文件时可不添加，**"ZW3D_DIR"**是值为 ZW3D 安装路径的环境变量。

4. 生成项目

右键点击项目，选择生成项目。您可以根据您的工程创建信息在工程目录 **"...\HelloZW3DAPI\x64\Release\HelloZW3DAPI.dll"**中找到"HelloZW3DAPI.dll"。

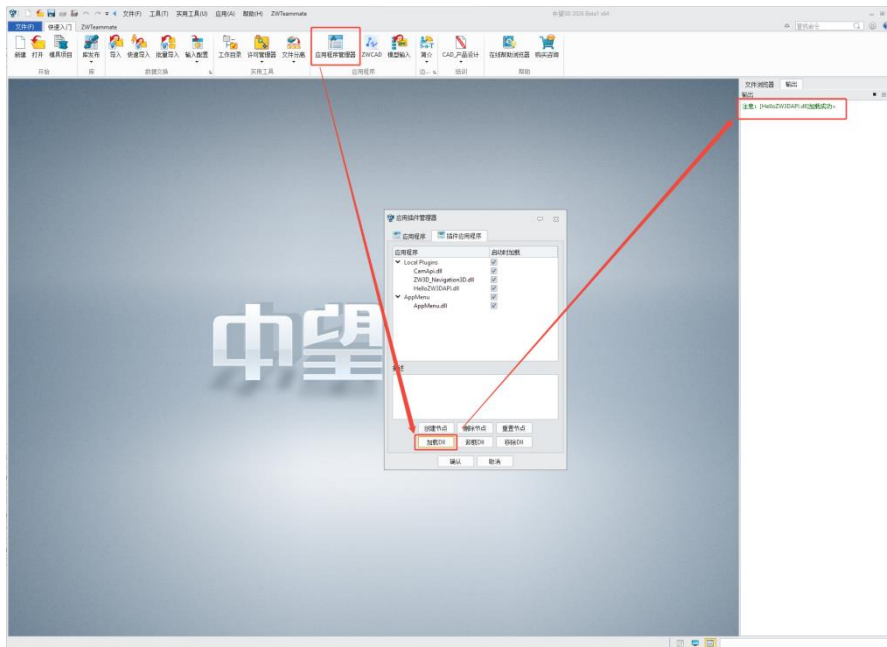
```
1>已完成代码的生成
1>HelloZW3DAPI.vcxproj -> D:\ZW3D\API_Training\HelloZW3DAPI\x64\Release\HelloZW3DAPI.dll
1>IF EXIST "D:\ZW3D\0206\zrc.exe" "D:\ZW3D\0206\zrc.exe" "D:\ZW3D\API_Training\HelloZW3DAPI\x64\Release\HelloZW3DAPI.zrc"
1>已完成生成项目“HelloZW3DAPI.vcxproj”的操作。
----- 全部重新生成: 成功 1 个, 失败 0 个, 跳过 0 个 -----
```

注意："Release"目录取决于您的项目配置，如果您用"Debug"生成项目，它将是"Debug"。

5. 加载 DLL 文件

方法 1: 使用 ZW3D 的 **应用程序管理器** 来加载这个 DLL 文件，输出对话框的信息将显示命令成功还是失败。

注意: 关闭 ZW3D 时，会将 DLL 的路径记录到注册表，再次启动 ZW3D 时将自动加载此 DLL。



方法 2: 复制 HelloZW3DAPI.dll 至用户目录下的"apilibs"文件夹，然后启动 ZW3D。

"%appdata%\ZW3D\ZW3D WuKong 2027\custom\apilibs"

注意: ZW3D 会自动加载"apilibs"文件夹下的应用程序。

方法 3: 在 ZW3D 的安装目录下创建"apilibs"文件夹，然后将 HelloZW3DAPI.dll 复制到该文件夹下，启动 ZW3D 同样会加载该文件夹下的应用程序。

方法 4: 注册并加载指定路径下的资源文件。

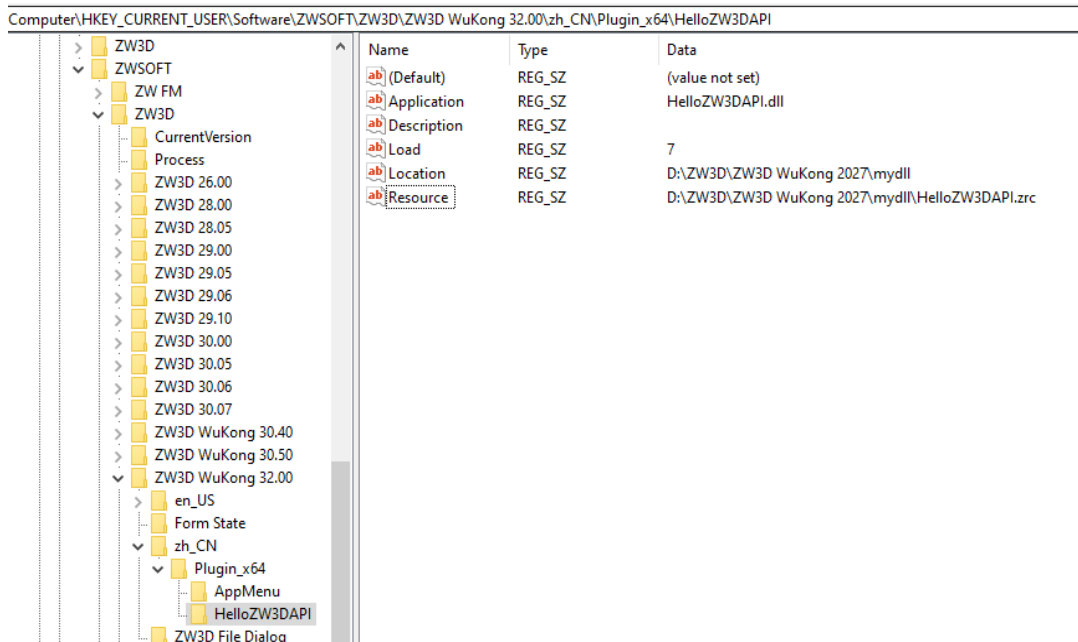
- 您可以将 HelloZW3DAPI.dll 及其他资源文件放在自定义的目录下，如
"...\Desktop\ZW3D\ZW3D WuKong 2027\mydll"。
- 同时您需要在插件初始化的过程中调用 **"ZwResourcePathAdd"** 注册搜索路径
"...\Desktop\ZW3D\ ZW3D WuKong 2027\mydll"。并且调用 **"ZwProfileModuleNameRegister"** 注册路径上的某一个路径名称如 **"mydll"**。
- 您需要在开启 ZW3D 前在注册表
"HKEY_CURRENT_USER\Software\ZW3D\ZW3D" 中添加资源信息，如 32.00 (ZW3D WuKong 2027) 版本的 ZW3D 对应插件注册表在 "ZW3D WuKong 32.00\zh_CN\Plugin_x64" 中，您需要创建一个新的项，并且为其添加如下的内容：
Application 为 dll 文件名称，对应您指定加载的动态库名称；
Description 为插件描述，在应用插件管理器中作为文本可见；



load 为插件加载标记，默认填写 7 即可；

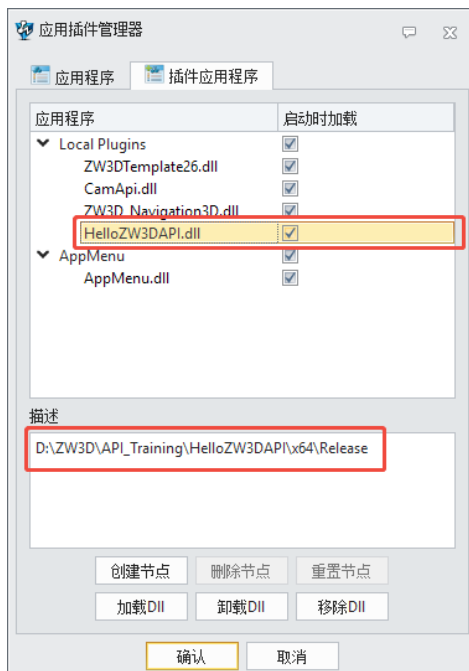
Location 为 dll 所在文件夹；

Resource 为创建模板命令时需要添加的 zrc 文件完整路径。



注意： 当您需要为单个 dll 注册多个关联的 zrc 文件时，您可以使用"**ZwResourcePathAdd**"注册需要的 zrc 文件，该界面支持通过绝对路径的方式加载 zrc 文件。

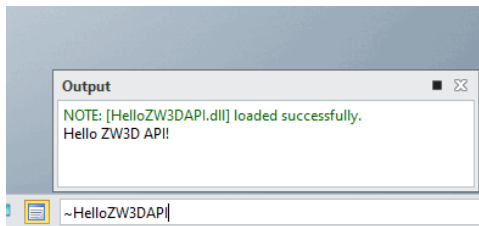
d) 配置完成后您可以看到 dll 文件被加载到插件管理器中。



注意： 开启 ZW3D 时会自动读取注册表中的相关信息，并完成对指定路径下插件及其资源文件的加载。修改注册表必须在未开启 ZW3D 的情况下进行，否则修改不会生效。

6. 运行应用程序

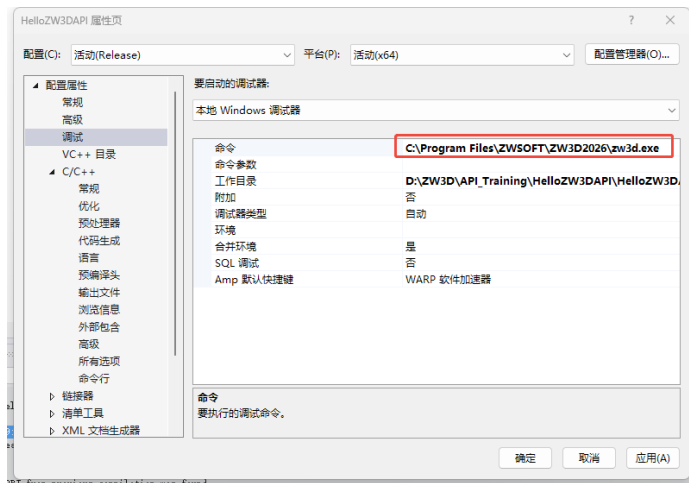
在命令行中输入"`~HelloZW3D API`"，按下 Enter。您会发现"`Hello ZW3D API!`"显示在输出对话框中。



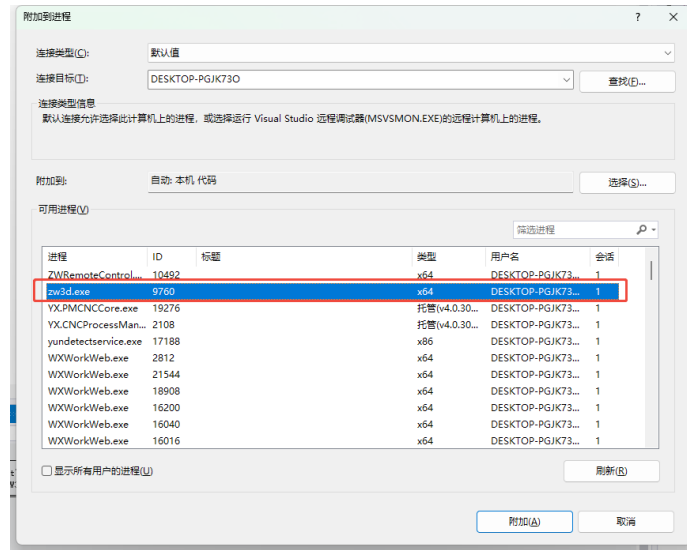
注意：函数名称可以定义为任何名称，但是为了防止出现相同命名的命令，**建议您在自定义名称时使用自定义的规范或者前缀，避免因命令名重复而注册失败。**您需要注意注册函数时的返回值，**如果该名字已经被使用了，接口会返回失败。**

7. 调试应用程序

方法 1：右键点击此项目并设置调试命令。运行本地 Windows 调试器或按 F5 启动调试。



方法 2：通过调试中的附加到进程将工程附加到已开启并加载 dll 文件的 ZW3D 上。



8. 为 main.cpp 设置断点

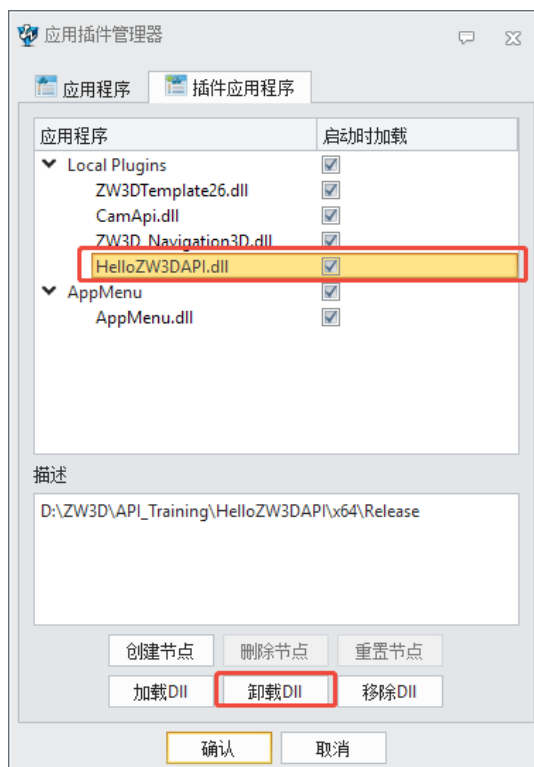
调试时您会发现有以下情况：

- 当加载 DLL 时，调试器会进入 HelloZW3DAPInit()。
- 当运行 "~HelloZW3DAPI" 等命令时，调试器进入其相关函数。



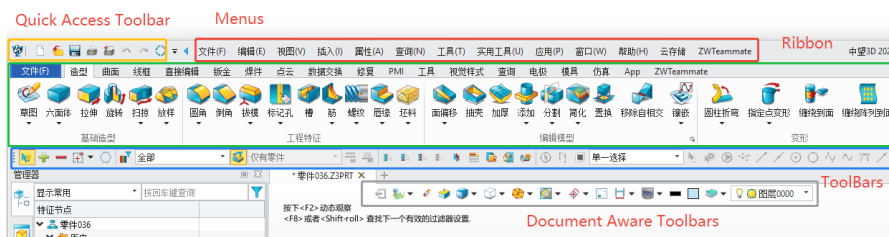
9. 通过应用插件管理器卸载 DLL

- 通过应用插件管理器可以卸载 DLL。
- 当卸载该应用程序时，调试器会进入 HelloZW3DAPExit()。



第三章 自定义菜单

虽然我们在 ZW3D 中创建了第一个自定义命令，但输入命令是相对烦琐的，创建一个可交互的按钮会更方便插件的使用。在这章我们会介绍如何将命令置于菜单（包括菜单、Ribbon 栏、工具栏等）。打开 ZW3D 自上而下依次可以看到快速访问工具栏、菜单、Ribbon 栏和工具栏，他们可以通过自定义进行编辑。



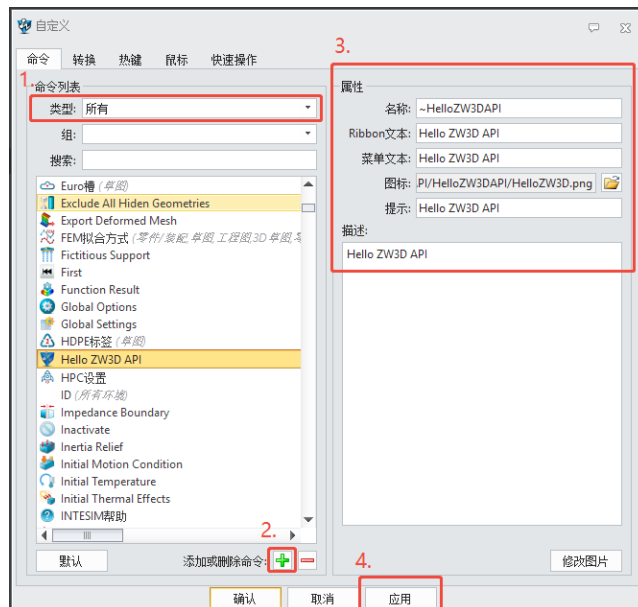
1. 打开页面管理工具

打开 ZW3D，右键点击 Ribbon 栏空白区域，然后点击自定义...



2. 自定义命令

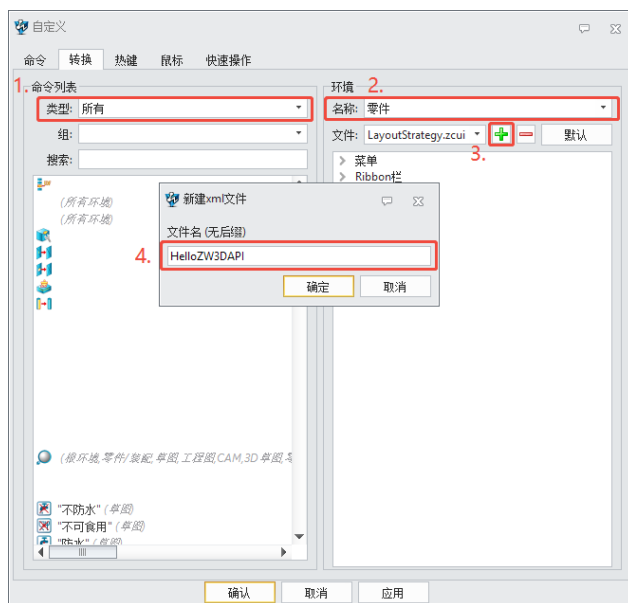
- 开启 **命令** 属性页。
- 将类型改成 **所有**。
- 点击 "+" 按钮添加新命令。
- 修改新命令的属性。
- 点击 "应用" 完成命令的创建。





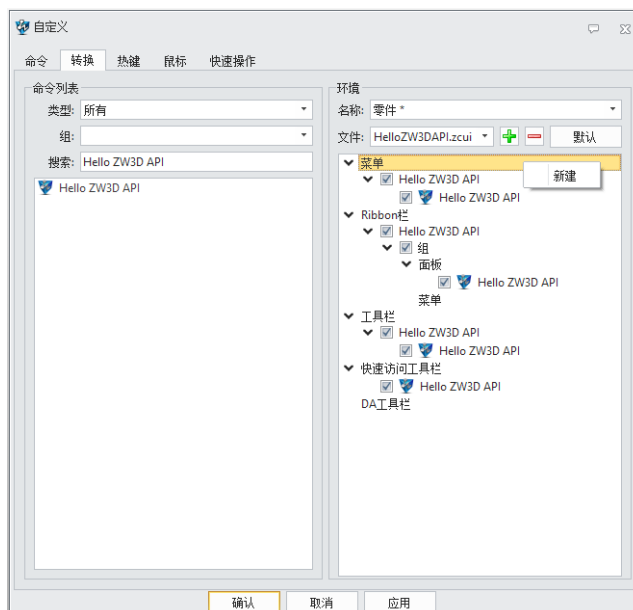
3. 创建您自己的菜单、Ribbon 栏、工具栏

- 切换至 **转换** 属性页。
- 将类型改成 **所有**。
- 将环境改成 **零件**（本例添加到零件环境，您可以根据自己需要的环境进行设置）。
- 点击 "+" 按钮，新建 zcui 文件。
- 设置文件名称为 "HelloZW3DAPI"。



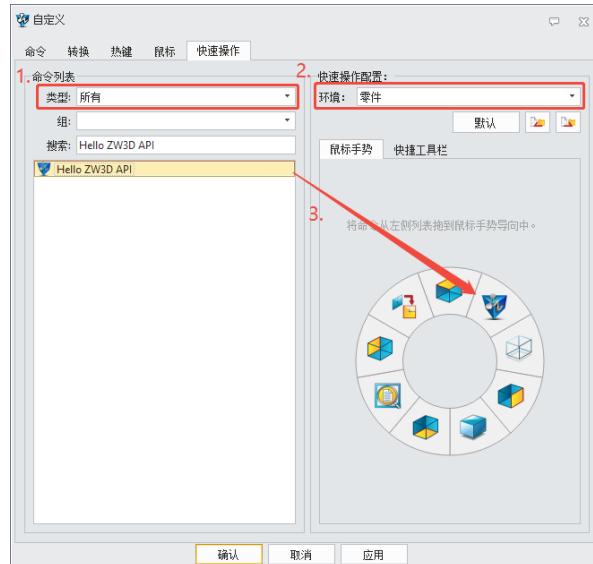
4. 自定义新 zcui 文件

- 右键点击菜单、Ribbon 栏、工具栏、快速访问工具栏，分别新建选项。
- 将命令 "Hello ZW3D API" 从左边拖拽至您刚刚创建的选项中。
- 创建如下图的界面布局。



5. 添加到快速操作

- 切换至 **快速操作** 属性页。
- 将类型改成 **所有**。
- 将环境改成 **零件**。
- 将命令"Hello ZW3D API"从左边拖拽至鼠标手势或快捷工具栏中。

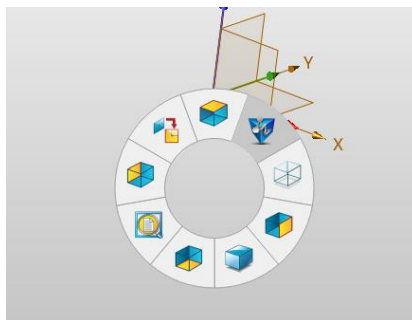


6. 点击确定完成自定义

- 创建零件环境，您能够在菜单、Ribbon 栏、工具栏的指定位置找到您添加的按钮，按下该按钮可运行命令"~HelloZW3DAPI"的命令。



- 按住"shift"+鼠标右键（快捷键根据您的设置）呼出鼠标手势，同样可以选择和执行此命令。

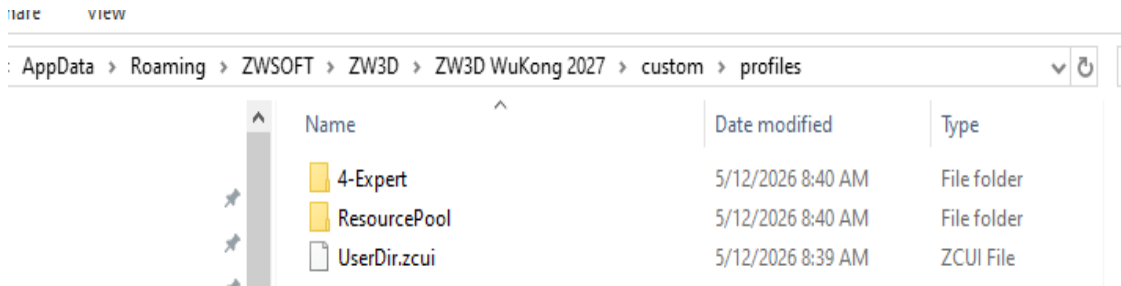


注意：如果您看到提示，"**ALERT: Unable to find HelloZW3DAPI: No such symbol.**"这意味着在 ZW3D 里找不到您的命令，您需要先加载 DLL。详情可参考第二章步骤 5 加载 DLL。



7. 部署 zcui 文件

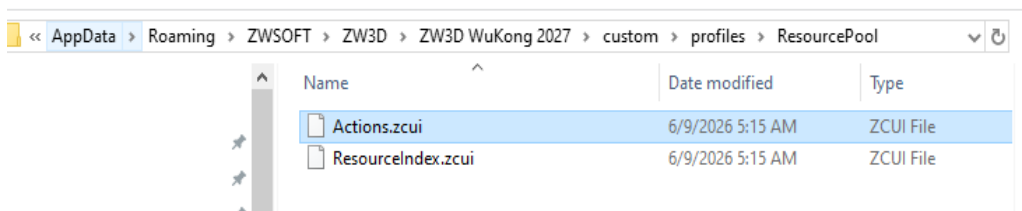
前文操作将会在"%appdata%\ZWSOFT\ZW3D\ZW3D WuKong 2027\custom\profiles"的路径下生成相关资源和配置的 zcui 文件，您可以通过复用这些文件实现自定义界面的配置。



2025 版本之前的 ZW3D 会生成旧版本的 zcui 文件，如果想要复用到 2025 之后的 ZW3D 需要对 zcui 文件进行升级之后才能进行配置，升级方法参考第 8 小节。

a) 从用户文件夹的路径下获取资源池 zcui 文件(Actions.zcui)。

通过前文的界面交互新增完您的自定义命令后，您可以在 ZW3D 的用户目录文件夹"%appdata%\ZWSOFT\ZW3D\ZW3D WuKong 2027\custom\profiles\ResourcePool"下找到名为"Actions"的 zcui 文件。



打开该文件您可以看到您的自定义命令文本被保存到了"Action"文件中。其中索引值"prefixKey"和"name"共同完成了对命令的唯一定位，通常情况下请不要修改索引值。

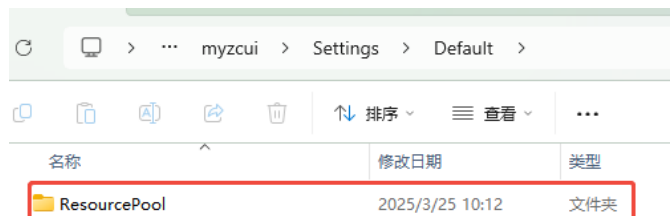


在 ZW3D2025 之前的版本中，您需要将文件拷贝到指定路径才能生效，现在您可以指定自定义的存放路径了，考虑到文件管理的便利性，更推荐您使用自定义文件路径。当您在"...\\Desktop\\myzcui\\"目录下存放文件时，您需要在该路径下创建到"...\\Desktop\\myzcui\\Settings\\Default\\"的基本文件层级，需要注意的是 Settings 和 Default 这两层是必要的识别路径，"myzcui"为您的自定义文件夹。在后文中您将了解到如何注册

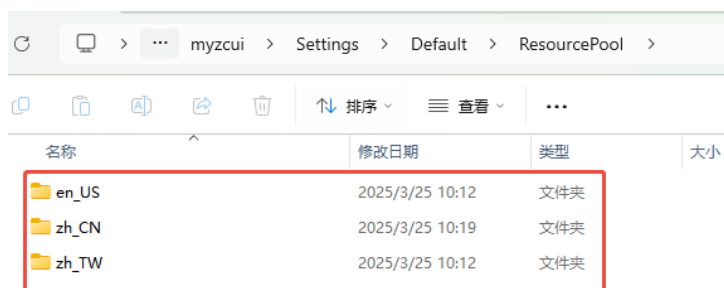


您的自定义路径。

"Action"文件需要您在"Default"文件夹中添加名为"ResourcePool"的子文件夹来进行存放，直接存放在该文件夹下的 zcui 文件在 ZW3D 启动时会被全量地加载。对于多语言的通用的配置您可以省略创建语言环境文件夹，但是当您用到多语言的设置时请务必分别创建对应的语言环境子文件夹。



当您需要自定义配置多语言环境下同个命令的不同文本时，您需要在"ResourcePool"文件夹下创建新的针对语言环境的子文件夹。参考左侧支持的语言表，您可以创建所需的语言子目录并配置相应的资源文件。需要注意的是直接放在"ResourcePool"下的通用配置与多语言配置不能存在完全相同的索引值"prefixKey"和名称"name"，否则可能会出现设置失效。因此我们不建议您在需要多语言配置的场合中直接把文件放在"ResourcePool"下，您可以根据语言环境的数量拷贝多份配置文件并分别放到对应子文件夹中。



区域	捷克	德国	英国	西班牙	法国	匈牙利	意大利	日本
语言	cs_CZ	de_DE	en_US	es_ES	fr_FR	hu_HU	it_IT	ja_JP
区域	韩国	波兰	葡萄牙	俄罗斯	土耳其	中国	中国台湾	
语言	ko_KR	pl_PL	pt_PT	ru_RU	tr_TR	zh_CN	zh_TW	

多个语言文件夹下的文件只会跟随当前启动的 ZW3D 语言环境进行加载，它们之间是可以存在完全相同的索引值"prefixKey"和名称"name"，此时您需要修改的内容仅有图中所示的文本部分。

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <Actions system="false">
3   <Action#List prefixKey="{a23af2bb-2900-408c-801f-a5cc795c4757}">
4     <Action name="ID_Action1" type="button">
5       <Form />
6       <Text>
7         <Ribbon>你好 ZW3D API</Ribbon>
8         <Menu>你好 ZW3D API</Menu>
9       </Text>
10      <Icon>D:/ZW3D/API_Training/HelloZW3DAPI/HelloZW3DAPI/HelloZW3D.png</Icon>
11      <Hint>你好 ZW3D API</Hint>
12      <Help />
13      <Desc>你好 ZW3D API</Desc>
14      <Script>~HelloZW3DAPI</Script>
15    </Action>
16  </Action#List>
17 </Actions>
18

```

分别将修改后的"Action.zcui"文件放到对应的子语言目录下，完成资源文件的放置。

名称	修改日期	类型	大小
Actions简体.zcui	2025/3/25 10:20	ZCUI 文件	1 KB

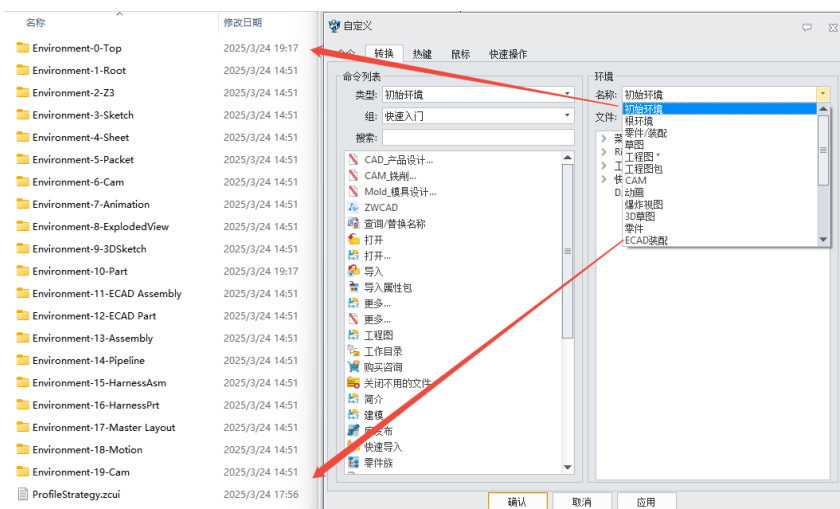
b) 从用户文件夹的路径下获取自定义策略的 zcui 文件(HelloZW3DAPI.zcui)

前文设置了零件环境下的自定义菜单，需要去对应路径

"%appdata%\ZWSOFT\ZW3D\ZW3D WuKong 2027\custom\profiles\4-Expert\Environment-10-Part"下找到名为 HelloZW3DAPI 的 zcui 文件。

名称	修改日期	类型	大小
HelloZW3DAPI.zcui	2025/3/24 19:21	ZCUI 文件	3 KB
LayoutStrategy.zcui	2025/3/24 19:21	ZCUI 文件	40 KB

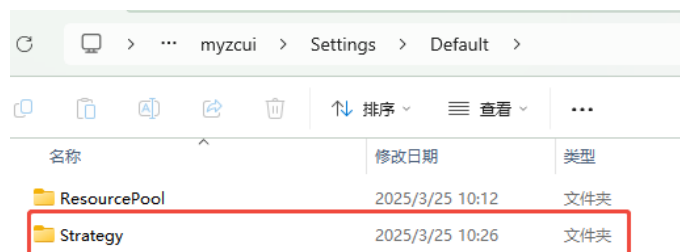
由于自定义的选项与环境的文件夹是一一对应的，当您文件设置在其他环境下时需要去对应路径查找。需要注意的是其中"Environment-2-Z3"对应的是多根文件环境的"Part\Assembly"项，而不是同时适配到单根的零件和装配两个环境下。



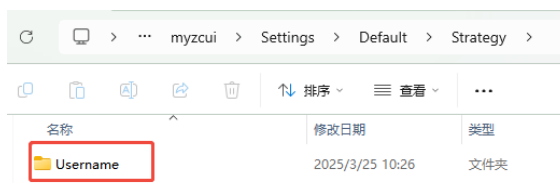
打开该文件可以看到"HelloZW3DAPI.zcui"适配的环境和角色分别是"10-零件"和"4-专家"。您的配置仅会在平台对应的环境和角色下生效，例如您在"10-零件"和"4-专家"下的配置不会在选择到"10-零件"以外的环境或"4-专家"以外的角色时生效



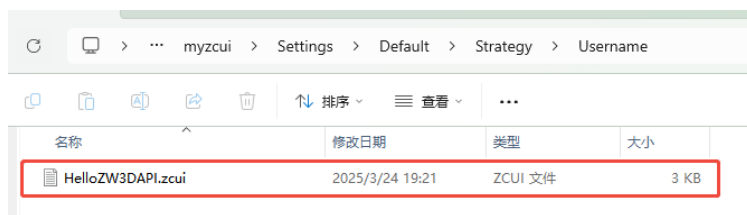
在前文创建的"...\\Desktop\\myzcui\\Settings\\Default\\"路径下添加 Strategy 文件夹。



由于原生的 zcui 策略保存在环境的子文件夹，且该文件夹参与 zcui 文件的识别，因此该文件夹是必要创建的（当不定义这层文件夹时无法找到 zcui 文件）。当然该文件夹的名称是支持自定义的，您可以自定义子文件夹的"UserName"。



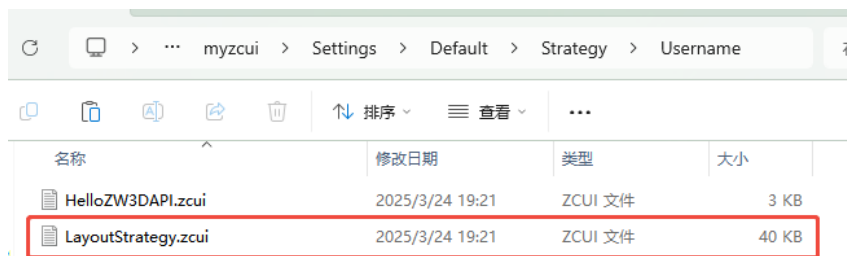
将修改后的"HelloZW3DAPI.zcui"文件放到自定义文件夹下，完成文件放置。



c) 从用户文件夹的路径下获取默认策略 zcui 文件(LayoutStrategy.zcui)

默认的用户差异化策略文件名为"LayoutStrategy.zcui"，对热键快速操作等内容的修改和配置会被写进这份文件中。当然您也可以在这份默认文件中修改菜单的配置，但是由于这份文件的唯一性，并不建议您在这份文件中直接配置菜单，如果您有必要的对热键和快捷手势可以修改并保存设置后，在对应路径，如上文的

"%appdata%\ZWSOFT\ZW3D\ZW3D WuKong 2027\custom\profiles\4-Expert\Environment-10-Part"中找到名为"LayoutStrategy"的 zcui 默认策略文件。



之后您可以将该文件复制到前文创建的自定义文件夹"..."

\Desktop\myzcui\Settings\Default\Strategy\UserName"下和前文拷贝的"HelloZW3DAPI.zcui"文件放到一起。由于 zcui 文件的生效是覆盖式的，当您在多份文件中对同个操作进行了不同的配置，此时后被加载的文件配置会覆盖掉先前加载的效果，因此请您注意您对自定义配置的唯一性。

d) 注册文件路径

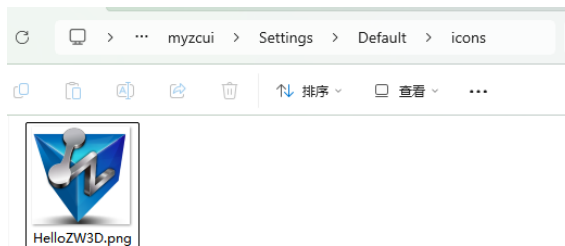
在前文中您创建了"... \Desktop\myzcui\Settings\Default\"基本层级目录，其中"Settings"和"Default"这两层是必要的识别路径，"myzcui"指代您的自定义文件夹。在该目录下您添加了"ResourcePool"和"Strategy"子文件夹用来放置资源文件和策略文件。

之后您需要在插件初始化的过程中注册搜索路径来实现资源自动加载。首先您需要调用接口"ZwResourcePathAdd"注册文件搜索路径"... \Desktop\myzcui\"。需要注意的是目录路径必须指向"Settings\Default"的上一级，即"myzcui"，否则搜索不到。然后调用接口"ZwProfileModuleNameRegister"注册路径上某一路径名称如"myzcui"。完成 zcui 文件路径的注册后，该路径下的 zcui 文件会在 ZW3D 启动时被全量地加载。如果 zcui 文件未被加载请检查路径注册是否未放在插件初始化的相关接口中。

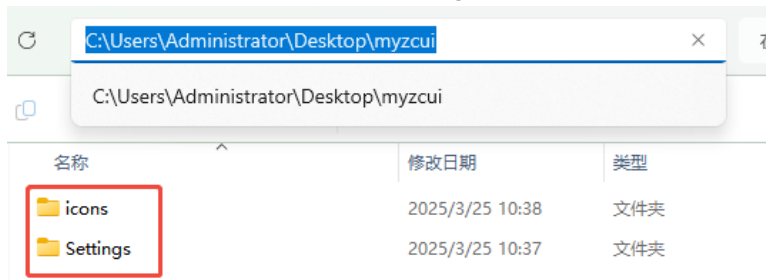
```
int HelloZW3DAPInit(int format, void* data)
{
    int nBytes = sizeof(zwString512);
    zwString512 path{};
    ZwPathSeparatorCheck("C:\\Users\\Administrator\\Desktop\\myzcui", nBytes, path);
    ZwResourcePathAdd(path);
    ZwProfileModuleNameRegister("myzcui");

    cvxCmdFunc("HelloZW3DAP", (void*)HelloZW3DAP, VX_CODE_GENERAL);
    return 0;
}
```

当您想要加载"...\\Desktop\\myzcui\\Settings\\Default\\icons\\HelloZW3D.png"作为图标文件时(自定义图标的类型仅支持 png 格式), 您需要调用"ZwResourcePathAdd"来注册图片所在文件夹的全路径, 如"...\\myzcui\\Settings\\Default".



当然, 您也可以将 icons 文件夹建于 Settings 相同层级来避免重复的路径注册。



注意: DLL 文件的初次加载后需要相应资源的更新, 因此直接加载 DLL 后您并不会直接看到 zcui 的配置生效, 需要您重启 ZW3D 或切换用户来完成资源更新, 为了保证资源的同步, 建议完成配置后重启 ZW3D。

您也可以按照第二章 5 的方法直接加载 dll 或放在 ZW3D 用户目录。

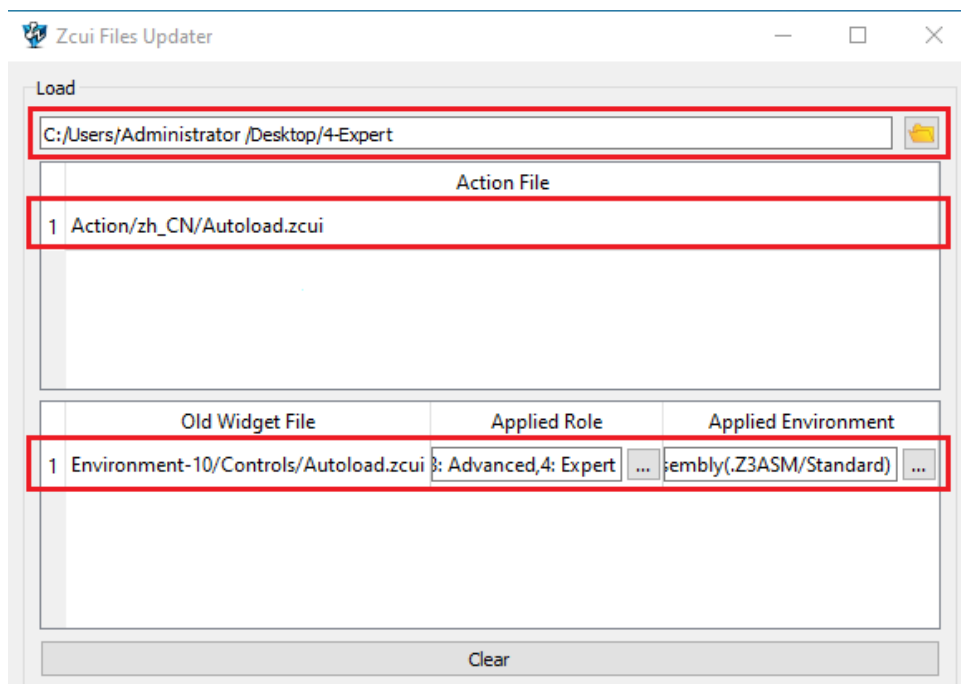
"%appdata%\ZW3D\ZW3D WuKong 2027\custom\apilibs"

用户目录下的资源文件会被自动加载。

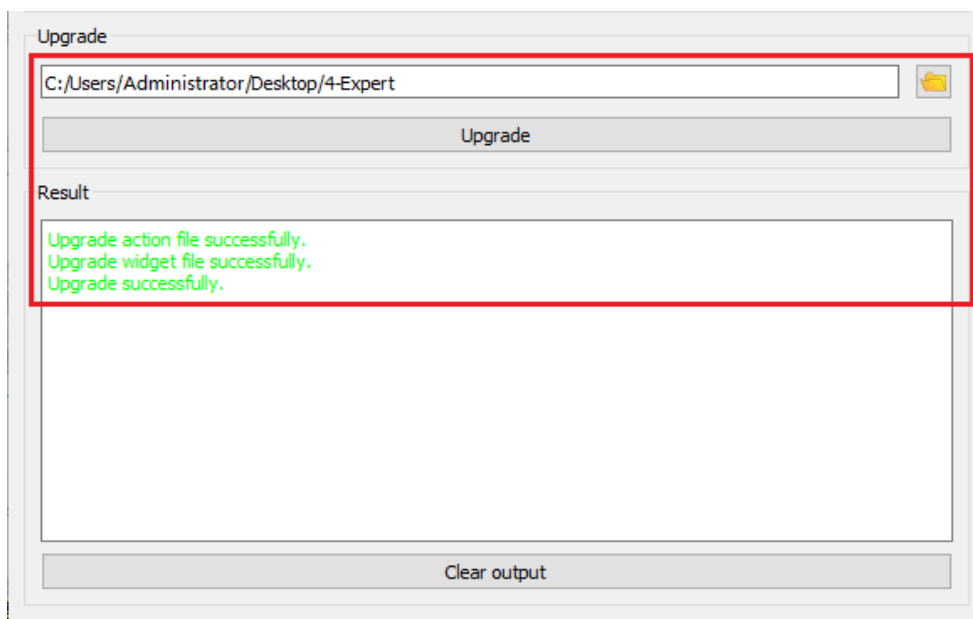
8. 升级 zcui 文件

zcui 文件在 ZW3D2025 版本存在较大的变动, 当您想要从低版本的 ZW3D 上迁移或复用已有的 zcui 文件到 2025 及以后版本时, 您需要先对其进行升级才能进行后续的配置和使用。为了方便您的使用, 我们在安装目录中提供了操作简便的 zcui 文件升级工具"ZcuiUpdater.exe"来帮助您升级来自旧版本的 zcui 文件。

- 添加您想要升级的文件所在路径, 小工具会自动加载路径下需要升级的低版本 zcui 文件。点击 clear 可以清空当前选择, 支持您重新进行选择。



- b) 选择升级后文件的输出路径并点击升级，当您看到绿色的提示升级成功的文字时文件升级成功，如果想要撤销本次升级请点击下方的清空输出。



- c) 升级完成后您可以在您指定的输出目录下看到 ResourcePool 和 Strategy 两个文件夹。按照前文的方式放置到指定文件夹即可。



Desktop\4-Expert

Name	Date modified	Type
Action	1/25/2024 8:31 PM	File folder
Environment-2	1/25/2024 8:31 PM	File folder
Environment-10	1/25/2024 8:31 PM	File folder
Environment-13	1/25/2024 8:31 PM	File folder
myzcui	1/25/2024 9:34 PM	File folder
ResourcePool	1/25/2024 9:27 PM	File folder
Strategy	1/25/2024 9:27 PM	File folder

注意: 从低版本升级来的 **zcui** 文件与 2025 版本创建的 **zcui** 文件存在一些差异, 比如 2025 版本创建的 **zcui** 存在索引值 **"prefixKey"**, 而低版本升级过来的 **zcui** 不存在该字段。因此为了防止出现同名的命令导致效果被覆盖的情况, 建议您对自定义命令的名称进行修改。在 2025 版本, 我们不建议您直接复制 **zcui** 文件到用户目录下的 **profile** 文件, 这种方式并不利于您对 **zcui** 文件的管理。如果您想要延续使用放置到用户目录的方式, 特别是复用从低版本升级的 **zcui** 文件时, 您需要注意将文本中的 **"DefaultCustomizations"** 换成 **"UserCustomizations"**, 并将对应的 **"name"** 修改成其他不会重复的名称。特殊情况下, 您可以选择将 **zcui** 文件迁移到 ZW3D 安装目录的 **Settings** 文件夹下将您的自定义 **zcui** 文件视为默认启动项。从安全性和可控性的角度, 我们不建议您使用直接复制的两种方式。

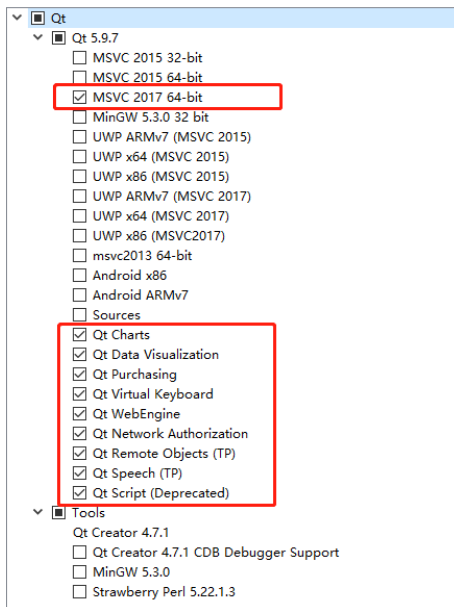


第四章 ZW3D UI 开发

1. ZW3D UI Designer 的概念

ZW3D UI Designer 是基于 QT Designer 的一个插件，我们基于 QT5.9.7 开发了自己的 UI 控件。用户需要自行下载安装 QT，并将 ZW3D 提供的插件拷贝到 QT Designer 相关目录下，启动 QT Designer，即可看到 ZW3D 定义的 UI 控件。具体操作如下：

a) 安装 QT5.9.7，在安装时注意勾选 Qt 相关组件。



b) 在 ZW3D 根目录找到以下 dll：

CommonControls.dll、QtnRibbon.dll

拷贝到 QT 的 designer.exe 同级目录。

c) 在 ZW3D 根目录下的"plugins/designer"文件夹内找到以下 dll：

CommonControlsPlugin.dll、QtnRibbonDsgn.dll

拷贝到 QT 的"plugins/designer"文件夹。

d) 在 ZW3D 根目录下的"plugins/styles"文件夹内找到以下 dll：

QtnStylePlugin.dll

拷贝到 QT 的"plugins/styles"文件夹。

注意：您只能将 ZW3D UI Designer 用于 UI 设计。所有编码逻辑均不能使用 QT 技术，ZW3D UI Designer 不包含 QT 库。因此如果您想使用 QT 功能，您需要购买商业版本。

2. 您也可以通过添加脚本的方式实现安装：

在 ZW3D WuKong 2027 版本安装路径的 api 档夹中创建一个"CopyQTDll.bat"文件，编辑文本并复制粘贴以下内容，注意修改 QTPath 到您本地的 Qt 路径。

```
%@echo off%
set CurrentPath=%~dp0
cd /d %CurrentPath%
```

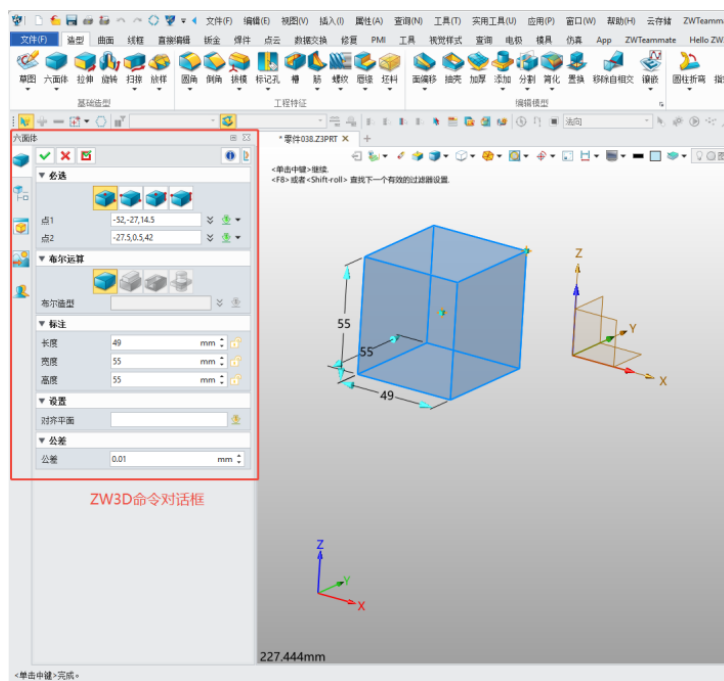


```
cd ..
set ZW3DPath=%cd%
if "%QTDIR%"==" " (
    set QTPath=D:\Qt\Qt5.9.7\5.9.7\msvc2017_64
) else (
    set QTPath=%QTDIR%
)
md "%QTPath%\plugins\styles"
COPY "%ZW3DPath%\CommonControls.dll" "%QTPath%\bin"
COPY "%ZW3DPath%\QtnRibbon.dll" "%QTPath%\bin"
COPY "%ZW3DPath%\plugins\designer\CommonControlsPlugin.dll"
"%QTPath%\plugins\designer"
COPY "%ZW3DPath%\plugins\designer\QtnRibbonDsgn.dll"
"%QTPath%\plugins\designer"
COPY "%ZW3DPath%\plugins\styles\QtnStylePlugin.dll" "%QTPath%\plugins\styles"
Pause
```

注意：该脚本仅适配 2025 及之后版本的 ZW3D，早期版本的配置文件存在差异，请参考对应版本的帮助文档进行设置。

3. ZW3D 命令对话框

ZW3D 命令对话框是用户在运行 ZW3D 命令时进行交互的特殊对话框。ZW3D UI Designer 支持特殊控件从 ZW3D 建模区获取必要的值，或者获取和设置只能在 ZW3D 使用的某些特殊值。

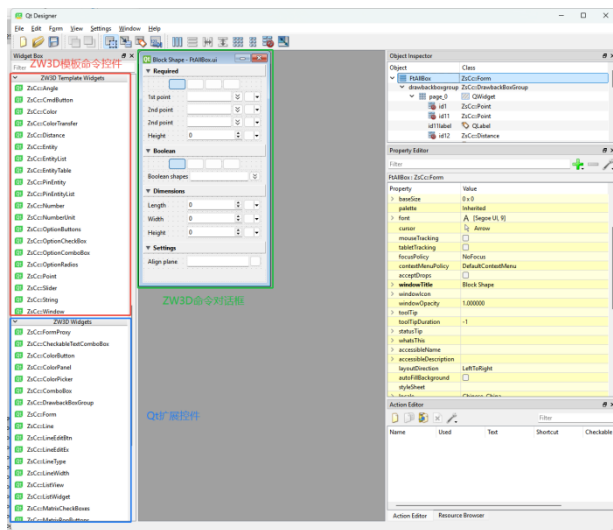


注意：如需下载所有的 ZW3D 命令对话框，请联系中望技术支持。



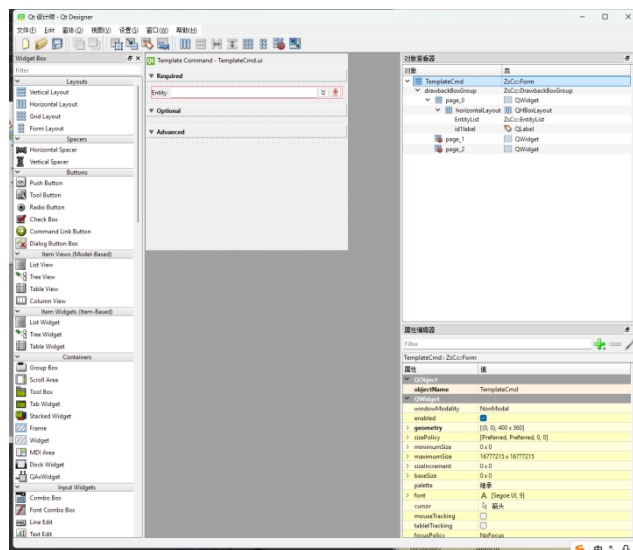
4. ZW3D UI Designer 功能区

由下图可知，您可以发现左边有两个扩展控件区域。其中红色标示框内的控件是 ZW3D 命令控件，用于 ZW3D 命令对话框（模板命令为主）的设计。蓝色标示框内的控件是基于 QT 封装的扩展控件，用于非模板命令对话框设计。生成的命令对话框可以在右侧进行预览。在创建命令对话框（模板命令）时，为避免出现适配问题，如无特殊要求，我们建议您不要使用 QT 控件。而在创建单独的表单时您可以使用这些拓展控件。

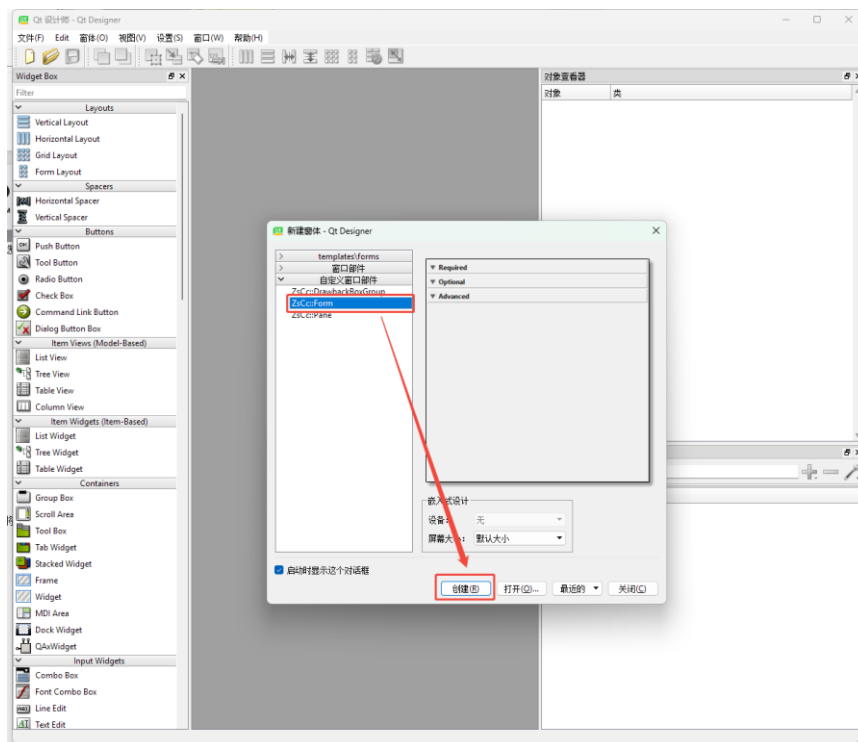


5. 创建 ZW3D 模板命令对话框

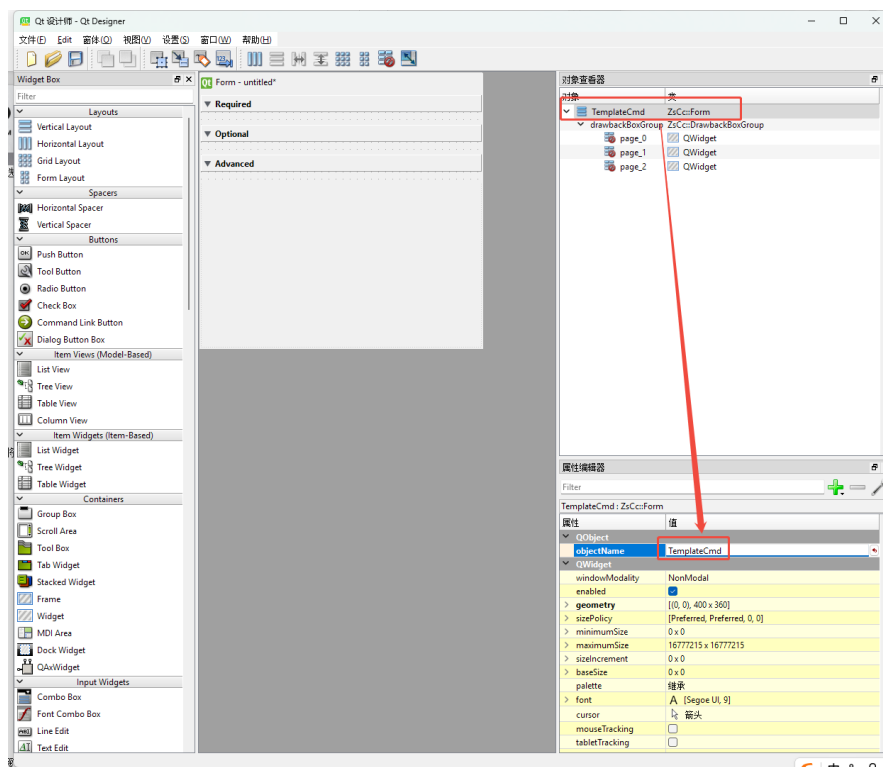
模板命令和非模板命令处理机制是不同的，模板命令由 UI 界面和对应的 Tcmd 控制文件共同实现控件机制及函数调用，其唤起格式为"!"+命令名称。而非模板命令没有处理的 Tcmd 文件，需要在 UI 上直接添加回调函数，并使用"~"+命令名称的格式调用。如果您设置参数错误的话将无法显示对应的命令窗口。相对于非模板命令，模板命令在功能复杂度和设计难度上都会更高，因此我们提供了一定的案例和说明来帮助您了解如何创建和编辑一个类似向导模板中的 "TemplateCmd.ui" 案例的 UI 文件，以及后续对 Tcmd 文件的编辑。



a) 选择基于"ZsCcc::Form"的窗口，并点击创建。

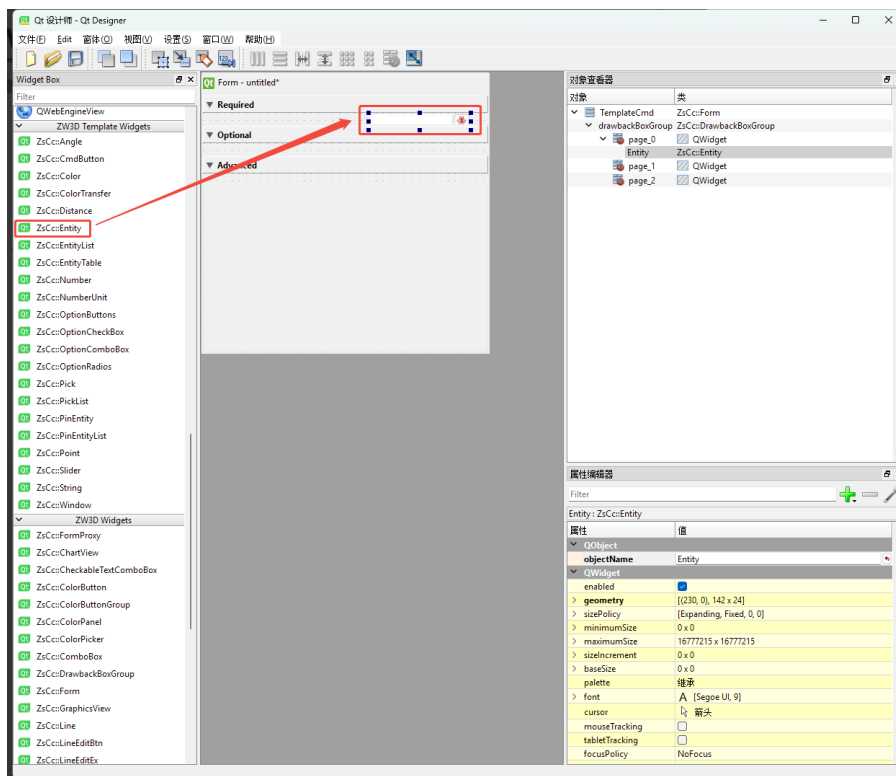


b) 将"form"的"objectName"设置为"TemplateCmd"。

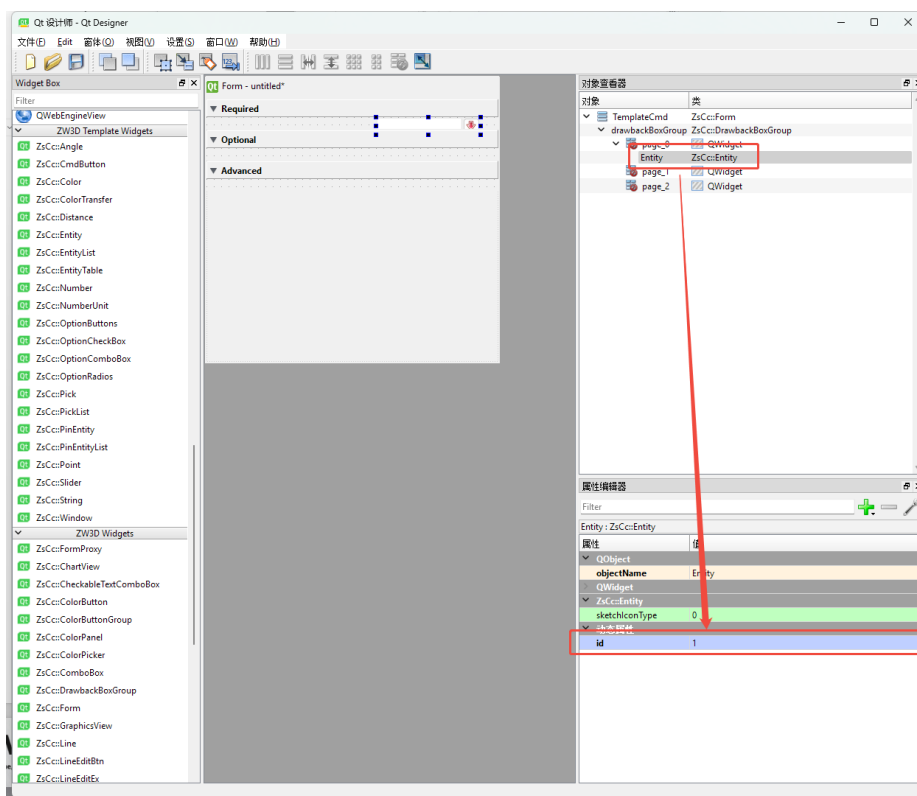


注意: UI 文件仅用来做页面的设计和布局，具体的功能设计需要在下文的 Tcmd 档编辑中实现。

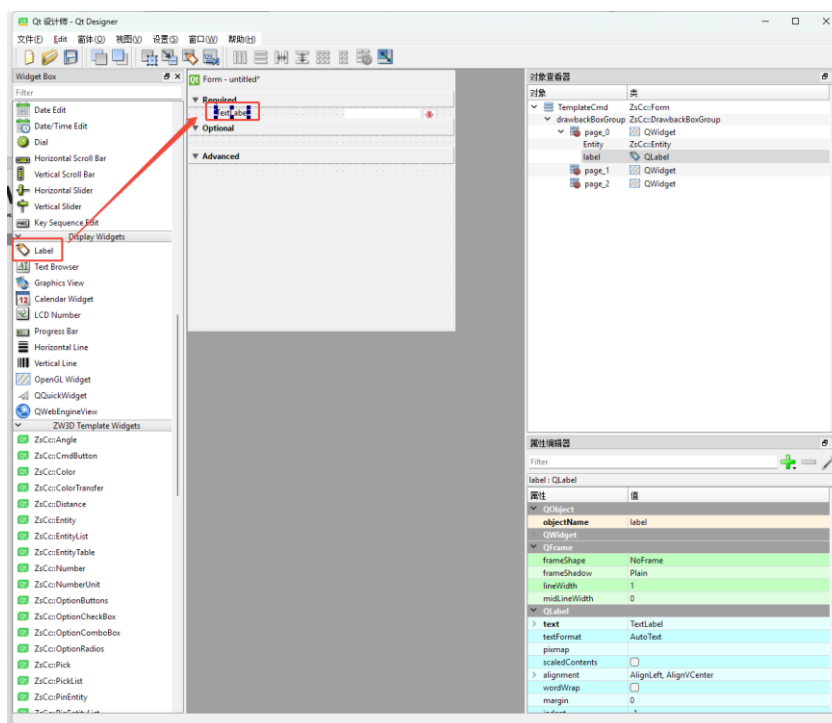
c) 将"Entity 控件"拖拽至 Required 区域中，完成控件的放置。



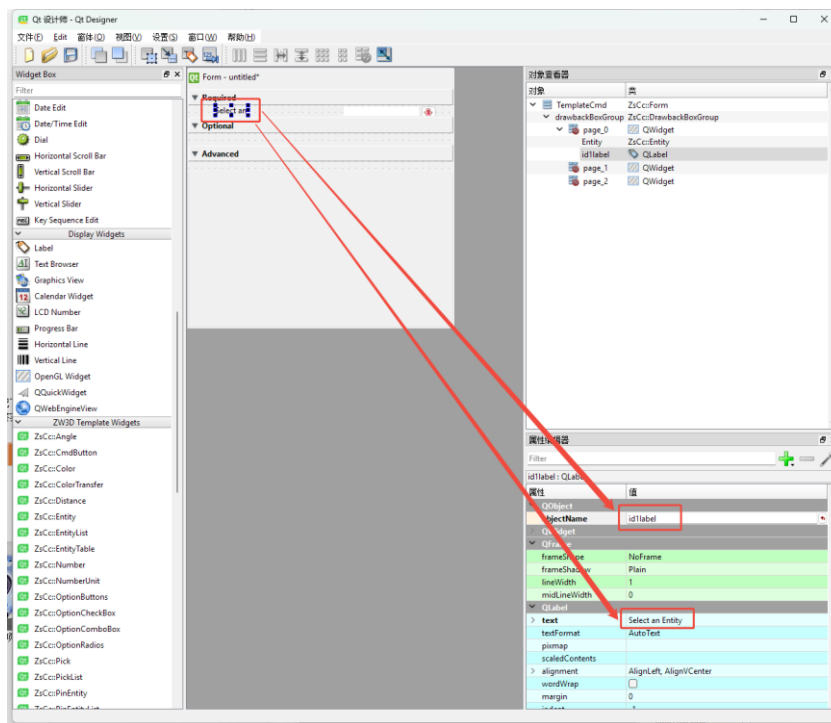
- d) 设置控件 id 为 "1"，该 id 为控件的识别标志，需要保证每个控件 id 的唯一性。



- e) 将"标签 Label"拖拽至模板的"Required"区域中，完成标签的放置。

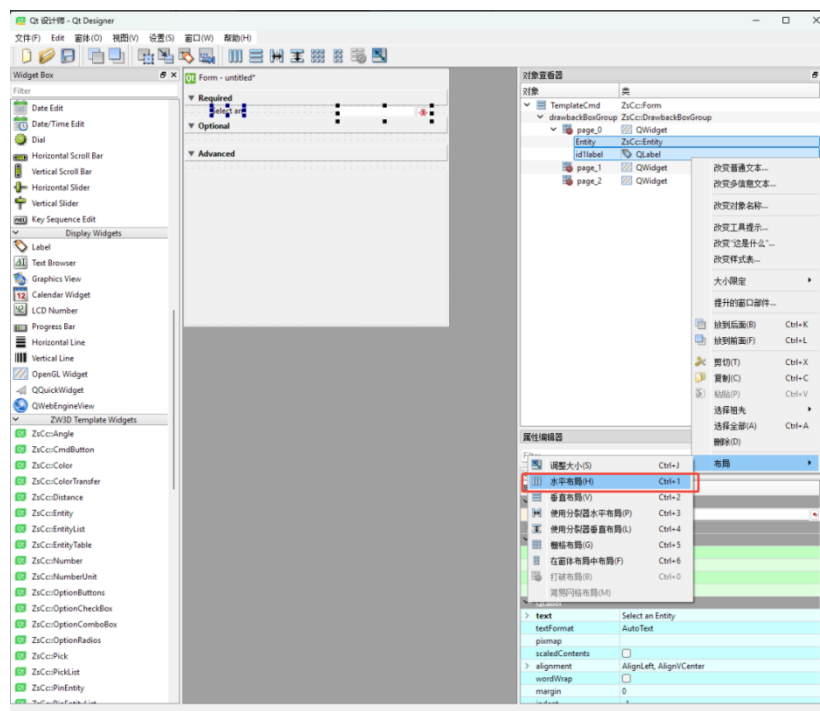


- f) 将该标签的文本改成"Select an Entity:", 并将"object Name"设置为"id1label".

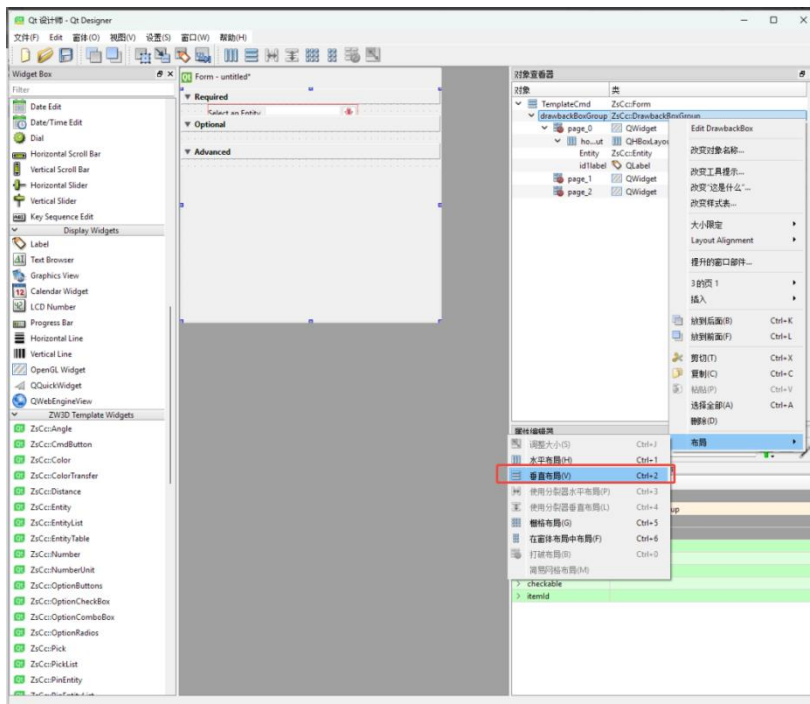


"Label"一般情况下是用来做文字指引和控制标记的，因此在有对应文本的情况下请按照"id"+"label"的格式来定义。另外一些 UI 界面对控件的操作也需要借助标签实现，因此请对标签进行规范性的命名。

- g) 按住"Ctrl"键并点选中 Label 和 Entity 这两个控件，打开右键菜单并进入布局选项，选择并执行水平布局。

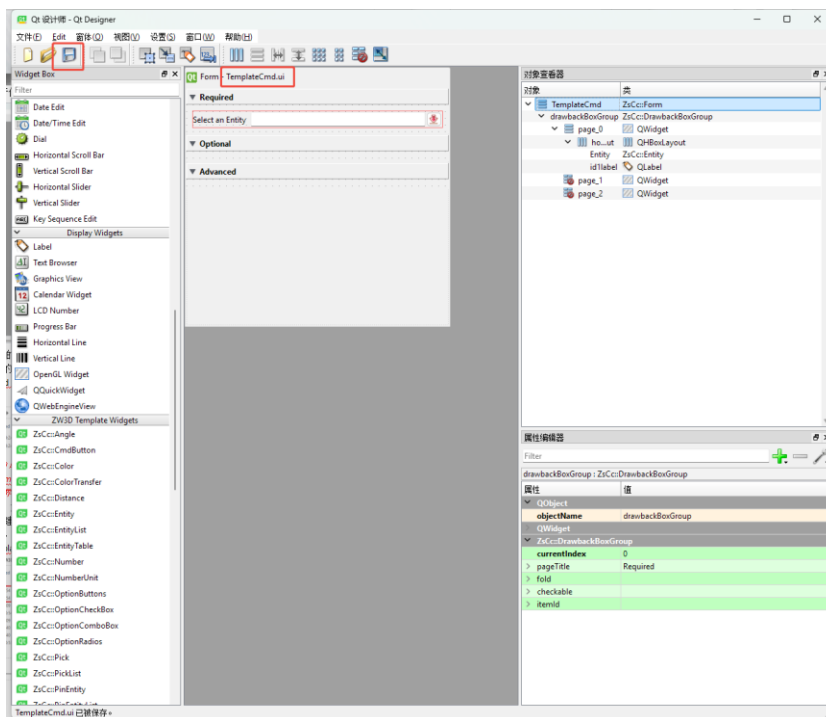


- h) 从右侧对象管理器或左侧面板中选中组别 drawbackBoxGroup，在控件焦点范围或面板范围内通过右键菜单进入布局选项，选择并执行竖直布局。



注意：当您鼠标的焦点未在对象范围内或者选错对象时可能无法唤起右键菜单，请尝试点击不同的对象或布局来达成您需要的布局效果。

- i) 将该对话框命名为"TemplateCmd.ui"并进行保存，完成 UI 面板的设计。



- j) 将 UI 文件导入 Tcmd 编辑器可以生成对应的 Tcmd 文件，通过对 Tcmd 文件进行进



一步的编辑来完成回调函数、控件属性等的设置。您可以参考或使用向导模板中提供的已经配置好属性的 `TemplateCmd.tcmd` 文件,根据第五章的内容来编辑 `Tcmd` 文件的内容。

Name	Date modified	Type	Size
TemplateCmd.tcmd	1/29/2024 10:24 PM	TCMD File	1 KB
TemplateCmd.ui	1/29/2024 10:24 PM	UI File	3 KB

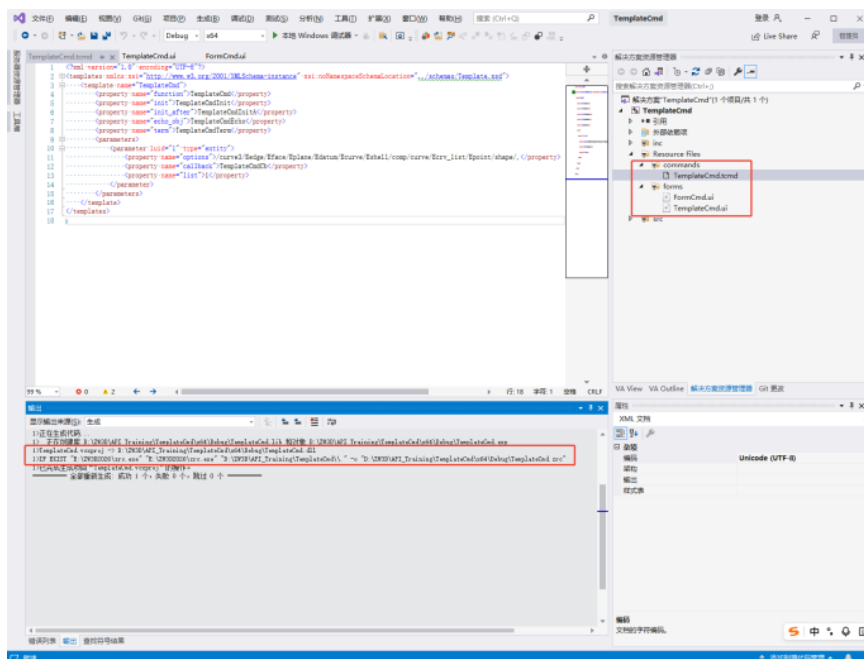
注意: 模板命令的文件名称需要与您注册的命令一致,例如您注册的模板命令为 `!TemplateCmd`,才可以正确地唤起 `TemplateCmd.ui` 和 `TemplateCmd.tcmd` 对应的界面,否则可能出现错误的匹配关系,或者无法唤起界面。

- k) UI 和 Tcmd 文件需要保持一定的层级关系,您可以直接将他们放在同个目录下。为了更好地管理资源文件,更建议您分别创建 `forms` 和 `commands` 去放置 UI 和 Tcmd 文件和命令文件,再将其添加到工程中。在模板中两个文件夹都是创建好的, `TemplateCmd.ui` 放在 `forms` 文件夹里, `TemplateCmd.tcmd` 放在 `commands` 文件夹里。

ZW3D > training > API_Training > HelloZW3DAPI > HelloZW3DAPI >

Name	Date modified	Type	Size
commands	1/25/2024 4:54 AM	File folder	
forms	1/25/2024 4:54 AM	File folder	
inc	1/25/2024 9:09 PM	File folder	
src	1/24/2024 10:54 PM	File folder	
x64	1/24/2024 9:09 PM	File folder	
HelloZW3DAPI.vcxproj	1/24/2024 9:40 PM	VC++ Project	9 KB
HelloZW3DAPI.vcxproj.filters	1/24/2024 9:40 PM	VC++ Project Filte...	2 KB
HelloZW3DAPI.vcxproj.user	1/24/2024 10:53 PM	Per-User Project O...	1 KB

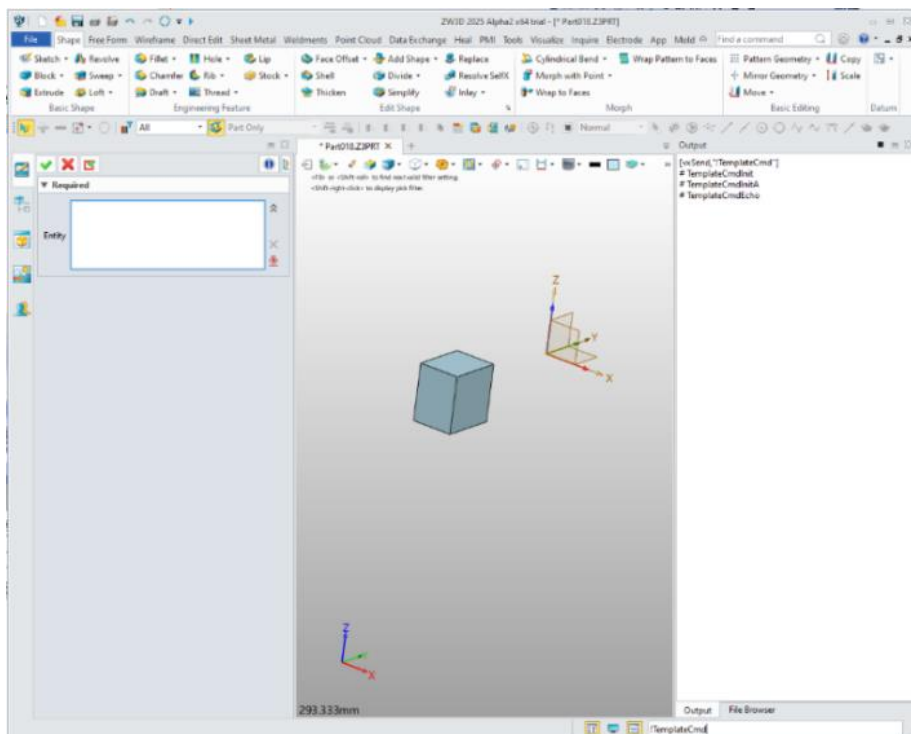
- l) 您可以参考二次开发向导模板 `"TemplateCommand.cpp"` 中的例子创建对应 `"!TemplateCmd"` 命令的执行函数。编译后生成资源的 `zrc` 文件。



- m) 加载 DLL 后,输入指令 `!TemplateCmd` 会根据 `Tcmd` 文件中的配置唤起对应的 UI 界



面。对于模板命令而言，UI 和 Tcmd 文件是相互关联且必需的。

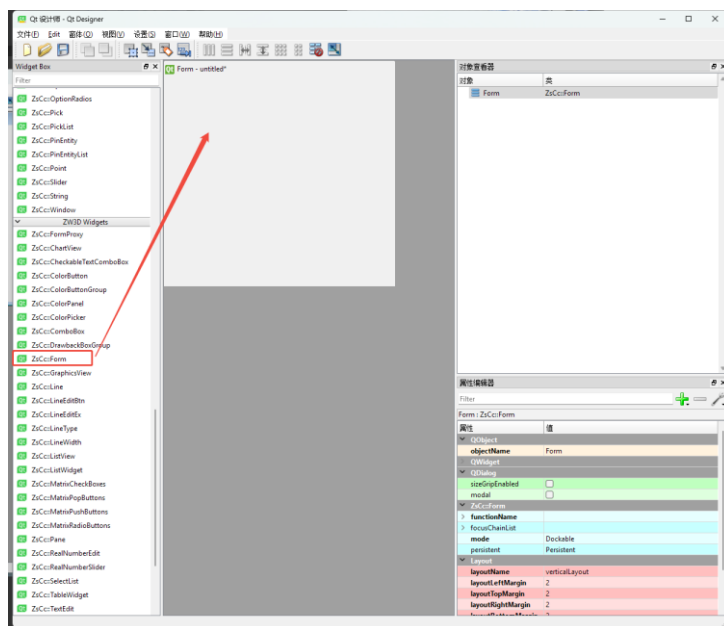


注意：模板命令的调用方式和非模板命令略有不同，**您需要在命令前加符号"!"而不是"~"**。具体的加载和调试可以参考第二章内容。请保证工程编译生成的 DLL 文件和 ZRC 文件在同一个文件夹下，来确保它们能够被加载到 ZW3D。

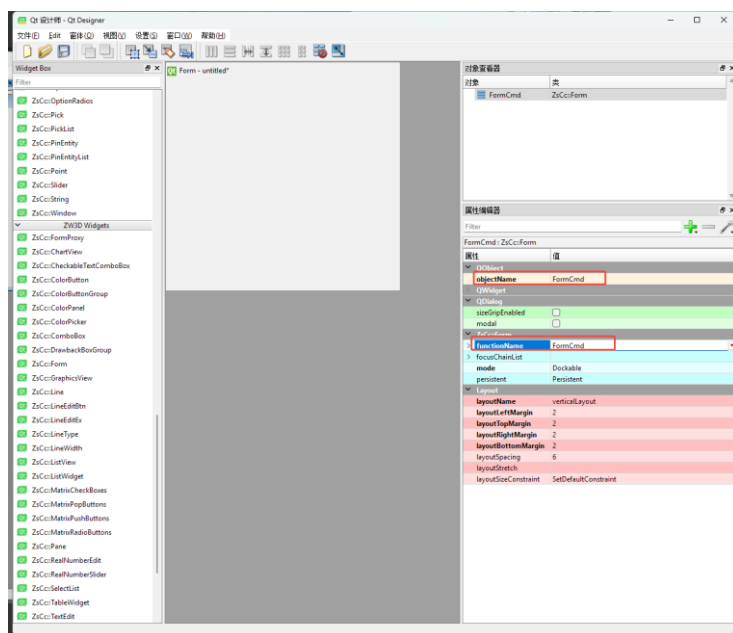
6. 创建 ZW3D 非模板页面

当您创建不使用 ZW3D 命令控件的非模板命令时不需要创建 Tcmd 文件。与模板命令不同的是，非模板命令用到的函数都是在工程文件中定义的，您需要在 UI 控件中添加回调函数，请根据自己的实际需要进行调整。向导模板中提供了名为"FormCmd.ui"的案例以及其代码文件"FormCommand.cpp"。

- a) 基于 ZsCc::Form 移除默认样式创建空的页面。当界面上存在多余的布局和控件时可以将其进行移除。

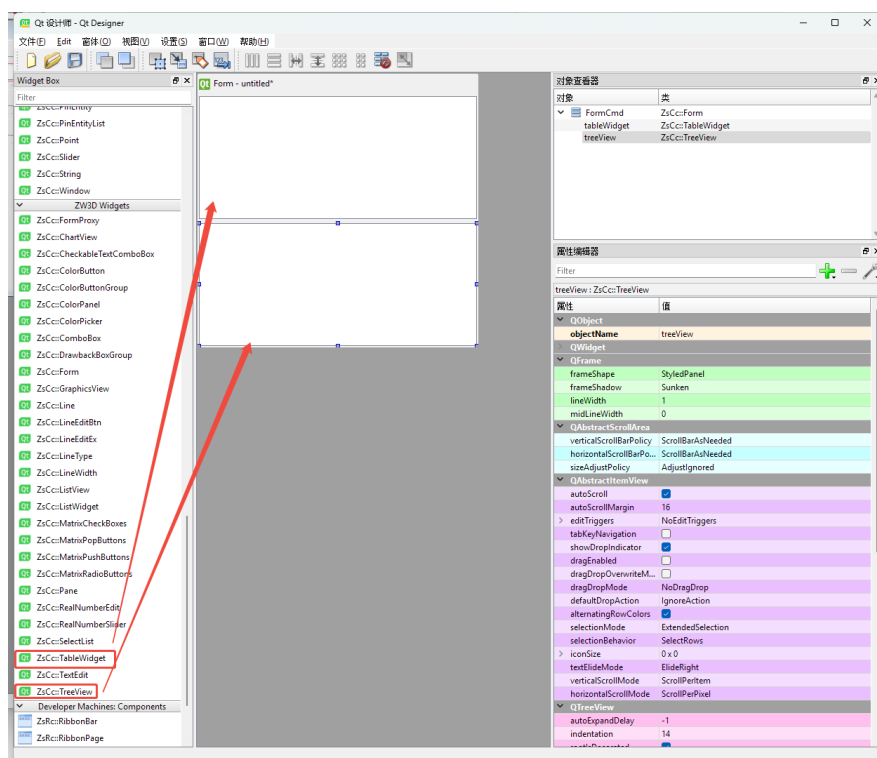


b) 设置其 `objectName` 和 `functionName` 都为 `FormCmd`。

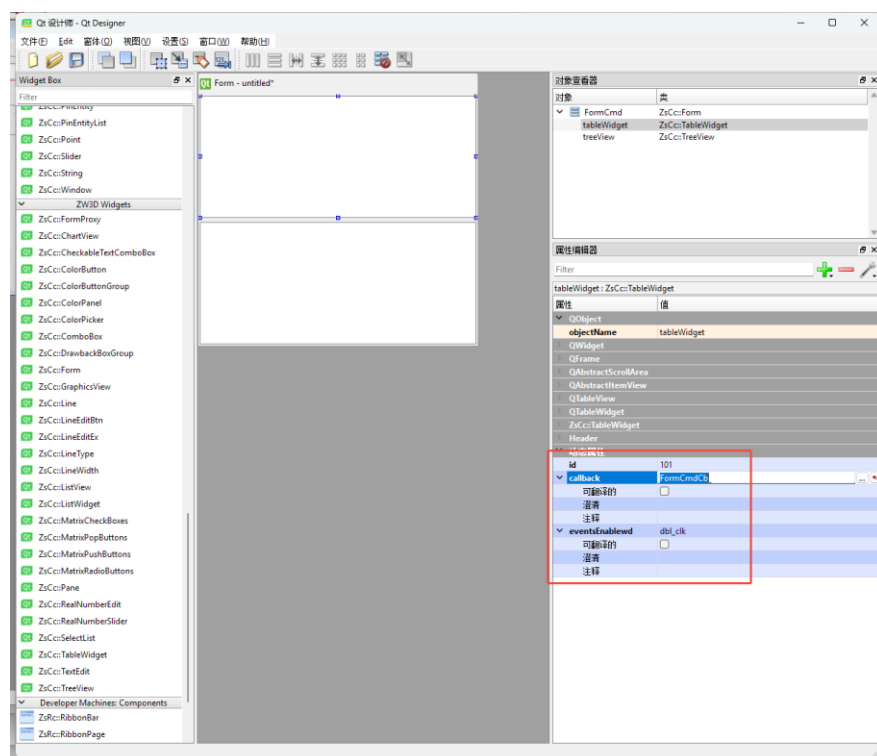


注意：该名称是识别非模板命令的关键，为了保证界面的正确调用，**请避免名称的冲突，确保其唯一性。**

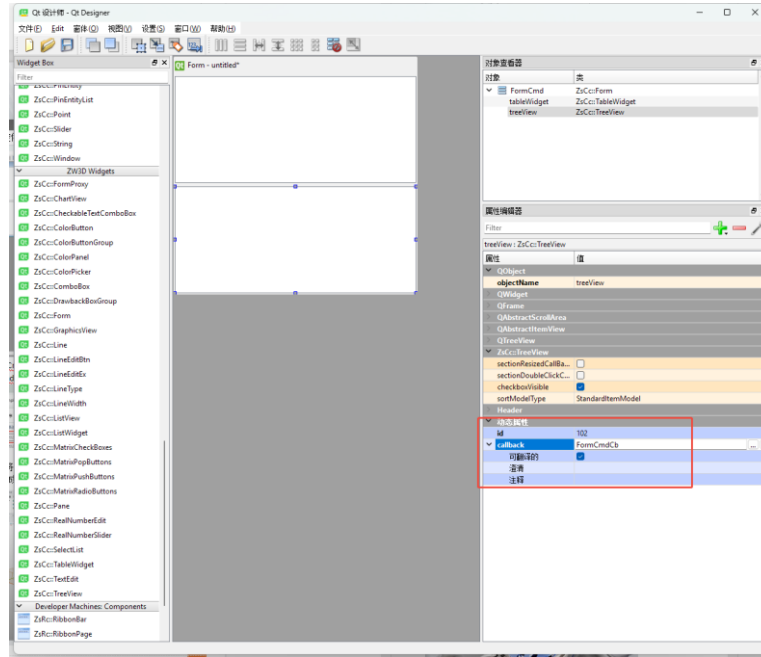
c) 将 `ZsCc::TableWidget` 和 `ZsCc::TreeView` 两个 Qt 扩展控件拖入面板中。



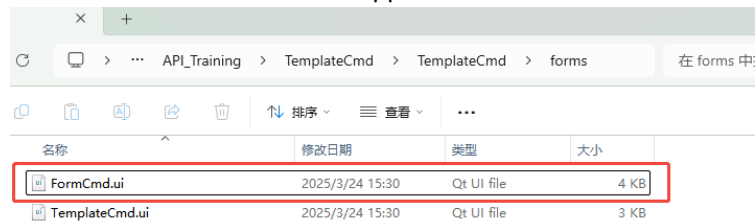
- d) 设置 TableWidget 的参数，设置控件 id 为 101，添加 callback 函数为 FormCmdCb，添加 eventsEnabled 并设置 dbl_clk 激活双击事件。



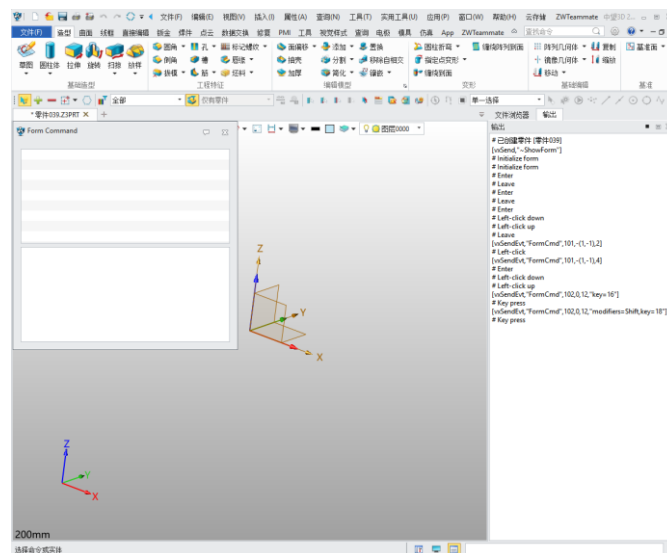
- e) 设置 TreeView 的参数，设置控件 id 为 102，添加 callback 函数为 FormCmdCb。



- f) 完成后将 UI 文件保存为"FormCmd.ui"并放到工程的 forms 文件夹里，您可以参考二次开发向导模板"FormCommand.cpp"中的例子创建和编辑对应的执行函数。



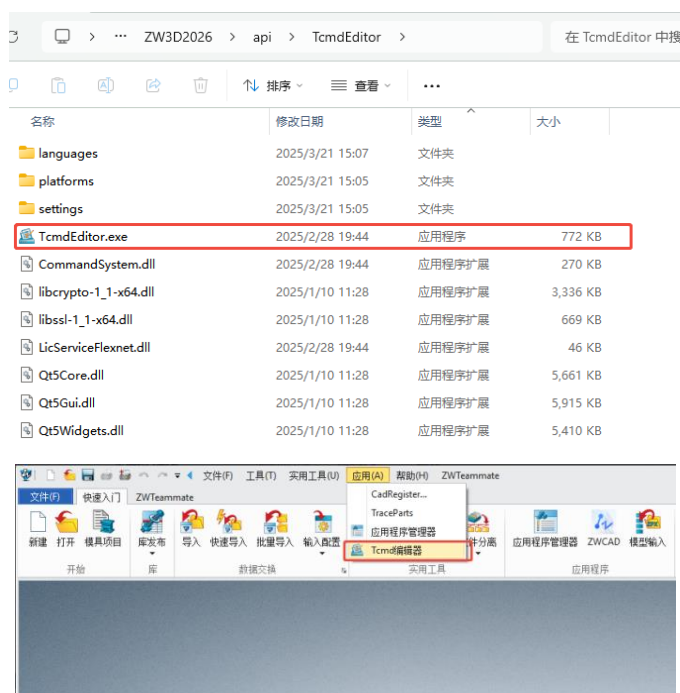
- g) 运行非模板命令"~ShowForm"将唤起对应的 UI 界面，值的变化会不断地触发回调函数。至此非模板的页面创建完成。



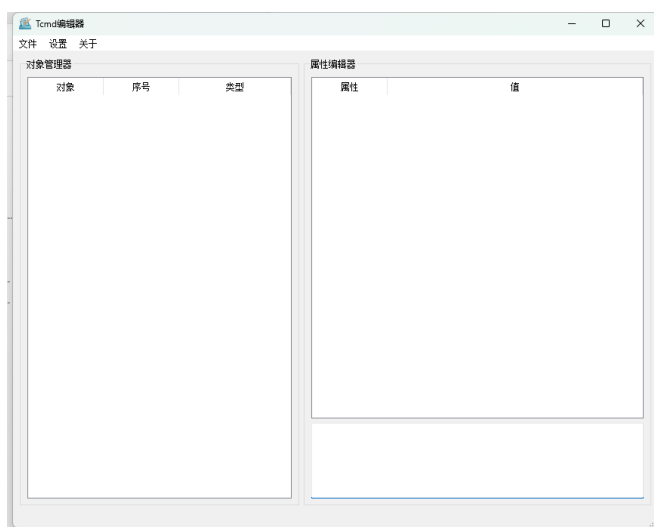
第五章 ZW3D Tcmd 开发

1. Tcmd 文件及编辑器

Tcmd 文件是与 UI 文件配套使用的控件配置文件，负责设置 UI 上指定控件的回调函数、过滤器等。我们提供了 Tcmd 编辑器程序进行 tcmd 文件的创建和编辑，该程序位于 ZW3D 安装路径下的"api\TcmdEditor"文件夹下，您可以在文件夹中直接打开 TcmdEditor.exe 或者通过 ZW3D 平台菜单栏应用中的链接来开启 Tcmd 编辑器。



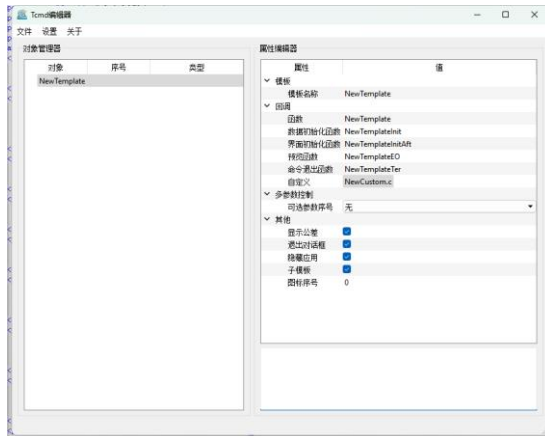
打开 Tcmd 编辑器交互面板，您可以使用它来更快捷地创建或编辑 Tcmd 文件。在本章中您将了解到如何使用 Tcmd 编辑器对 Tcmd 文件进行进一步的配置。



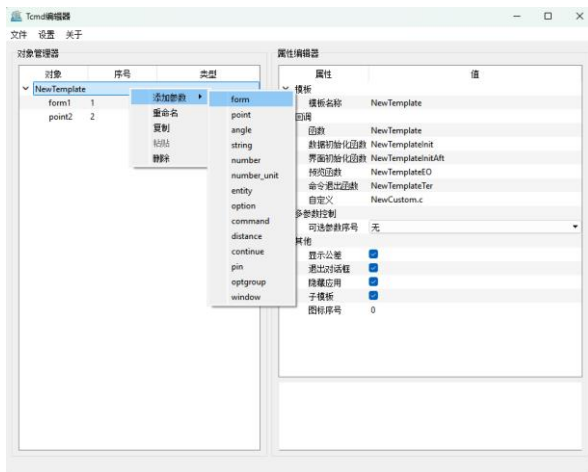


2. Tcmd 文件的构成

一个基本的 Tcmd 文件由窗口模板和参数控件组成，分别对应整个 Tcmd 文件的基本类型和各个参数控件的属性。使用 Tcmd 编辑器首先创建的就是窗口模板，您可以在左侧对象管理区通过右键添加空的模板对象，一份模板文件允许添加多个窗口模板。之后您可以根据需求在右侧的模板属性编辑区内对窗口模板的属性进行编辑。



最基本的模板由"模板名称"和"(执行)函数"组成，两者通常与命令保持同名。在 ZW3D 中通过"! +命令"的方式可以调用配置好的模板。一些回调函数的补充能够大大丰富模板的开发和使用。比如"预览函数"能够帮助您预览对象操作。"数据初始化函数"和"界面初始化函数"，分别可以在窗口构建前和构建后进行初始化。"命令退出函数"在命令退出时会被执行。"自定义"函数能够帮助用户自定义特征命令，在使用时"自定义"函数必须以.c 结尾，并且"(执行)函数"必须设置为"CustomOp"。"多参数控制"可以设置窗口下一些控件的显隐性。其他提供的窗口能力请参考 Tcmd 编辑器的帮助文档。



你可以右键指定窗口，通过弹出菜单为其添加各种参数，如实体（Entity）、表单（Form）、连续（Continue）、字符（String）、点（Point）、点云（Pin）、数（Number）、选项（Option）、命令（Command）、组（Optgroup）、窗口（Window）、距离（Distance）、角度（Angle）等等各类参数控件。每种参数控件都有其对应的属性，其中最基本的属性为类型（Type）和 Id，该值是默认设置的不能进行更改，一般情况下会在您导入 UI 时自动生成控件对应类型的参数。详细的属性介绍

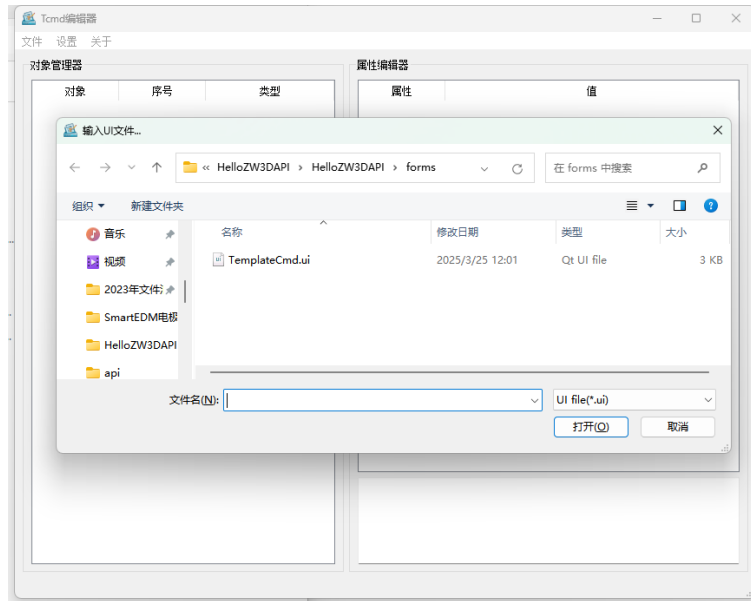


详细请参考 Tcmd 编辑器的帮助文档。

3. Tcmd 文件编辑案例

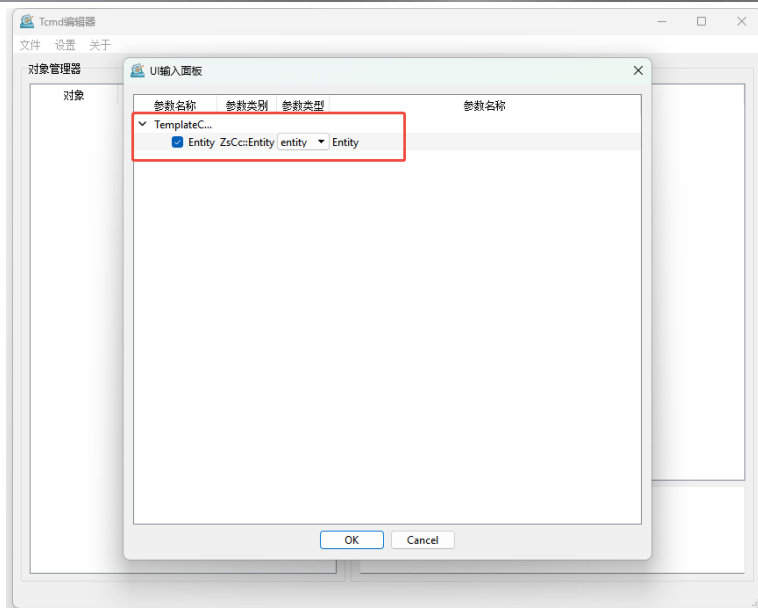
a) 找到 UI 文件

Tcmd 文件通常基于 UI 文件进行开发和编辑，使用 Tcmd 编辑器加载 UI 文件时，会检测同目录或父目录 **commands** 文件夹下存在对应 Tcmd 文件，如果找到会提示是否打开对应 Tcmd 文件，否则会创建新的 Tcmd 模板文件。找到前文提到的 "TemplateCmd.ui" 文件，并将其导入。



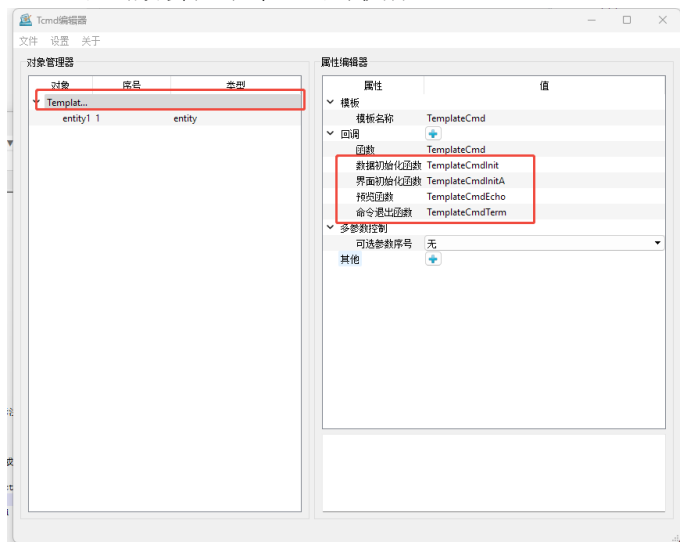
b) 根据 UI 文件加载或创建对应 Tcmd 文件。

加载上一章节创建的 "TemplateCmd.ui" 文件，当本地不存在已创建的 "TemplateCmd.tcmd" 时，编辑器会根据 UI 控件创建同名的 Tcmd 模板文件。Entity 类型的控件支持在创建时切换成 **point**，需要根据需求进行选择。点击 OK 完成 "TemplateCmd.tcmd" 文件的创建。



c) 编辑模板属性

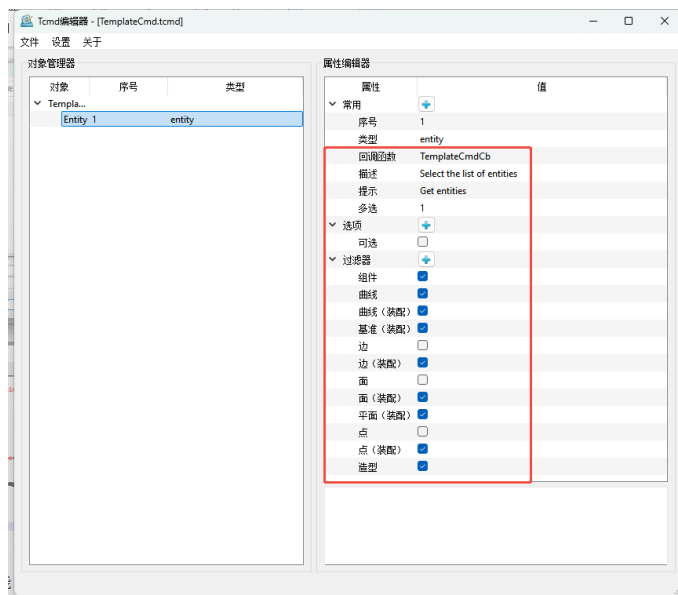
参考向导模板中的"TemplateCmd.tcmd"等文件为 Form 添加相关函数。填充函数为"TemplateCmd"，该函数为主要的执行函数。填充数据初始化函数为"TemplateCmdInit"，该函数会在界面创建前执行。填充界面初始化函数"TemplateCmdInitA"，该函数会在界面创建后执行。填充预览函数为"TemplateCmdEcho"，该函数会在界面预览流程执行。填充命令退出函数为"TemplateCmdTerm"，该函数会在命令退出时执行。



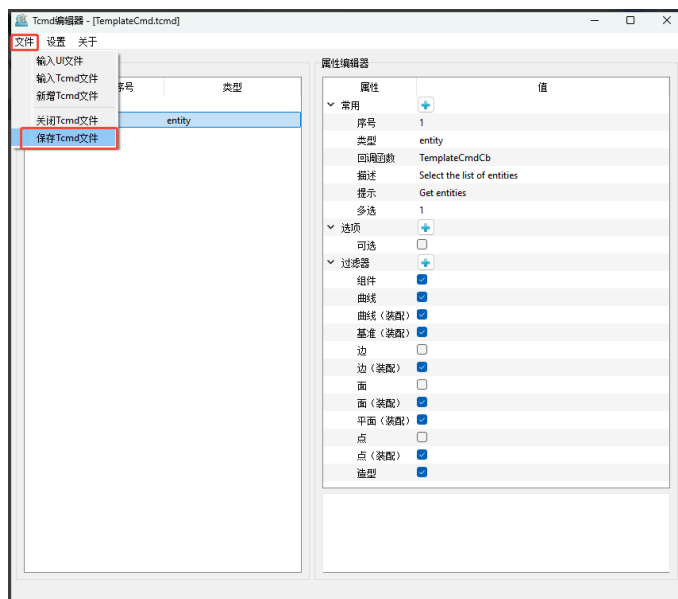
d) 编辑参数属性

为 entity 控件添加相关设置，具体的字段定义可以参考 Tcmd 帮助文档。序号和类型来自 UI 文件，为了保持 id 的一致性不支持对其修改。设置回调函数为"TemplateCmdCb"，当触发该界面上的该字段时，该函数将被执行。添加描述 Select the list of entities，该描述只在 Tcmd 中可见。添加提示 Get entities，该描述在界面左下角可见。添加过滤器选项为组件、曲线、曲线（装配）、基准（装配）、边（装配）、

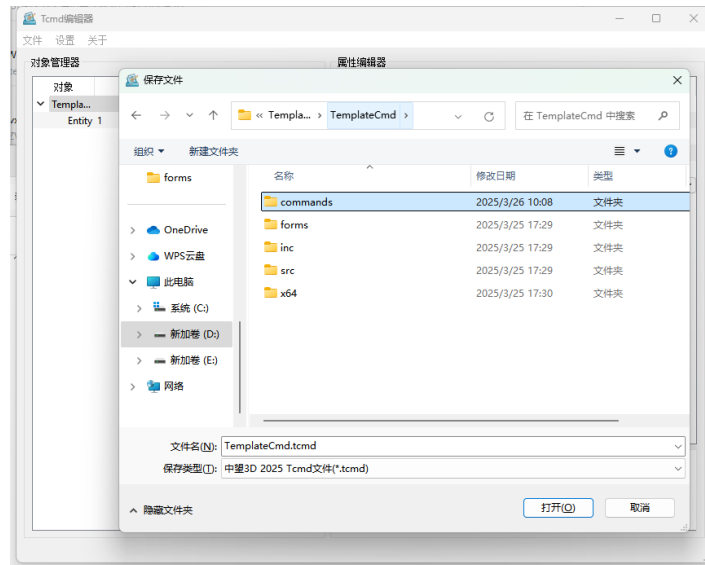
面（装配）、平面（装配）、点（装配）、造型。添加多选并且置 1，表示该控件支持多选。



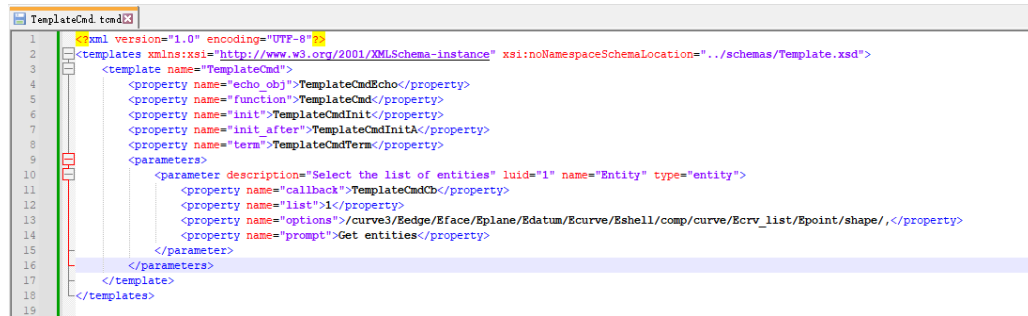
e) 点击文件下的保存 Tcmd 文件，完成对 Tcmd 文件的修改和保存。



f) 一般将 Tcmd 文件与 UI 文件保存在同目录或父目录 commands 文件夹下，这样在编辑和资源管理时可以更好地找到对应文件。



g) 找到并打开生成的"TemplateCmd.tcmd"文件，可以看到相关字段被填充到了文件中。



h) 补充实现函数

参考"TemplateCommand.cpp"为模板命令添加功能，补充模板命令的实现，使其能够修改指定实体的颜色。需要注意包含实体和面所需接口的头文件。

```
#include "zwapi_general_ent.h"
#include "zwapi_brep_face.h"
```

```
int TemplateCmd
(
    int idData    /* I: index of data container */
)
```

```
/*
```

```
DESCRIPTION:
```

```
    Execution function of command. the function is called when the command
    is clicked on the OK or APPLY button.
```

```
*/
```

```
{
    // TODO: Do something
    cvxMsgDisp("TemplateCmd");
}
```



```

/* Inquire data of data container */
int idField = 1;
int nEnts = 0;
int* idEnts = nullptr;
cvxDataGetEnts(idData, idField, &nEnts, &idEnts);
/* Change face color */
evxEntType type;
for (int i = 0; i < nEnts; i++)
{
    if (cvxEntClassNum(idEnts[i]) == VX_ENT_FACE)
    {
        svxFaceAt att;
        cvxMemZero(&att, sizeof(att));
        cvxPartInqFaceAt(idEnts[i], &att);
        att.front_color.r = 0;
        att.front_color.g = 0;
        att.front_color.b = 255;
        cvxPartSetFaceAt(1, &idEnts[i], &att);
    }
}
cvxMemFree((void**)&idEnts);
return 0;
}

```

i) 编译工程

同第二章编译后在指定路径下会创建 DLL 和 ZRC 等文件，需要注意的是 DLL 和 ZRC 应在同目录下并保持同名，来保证 ZRC 文件正确地被加载到 ZW3D。

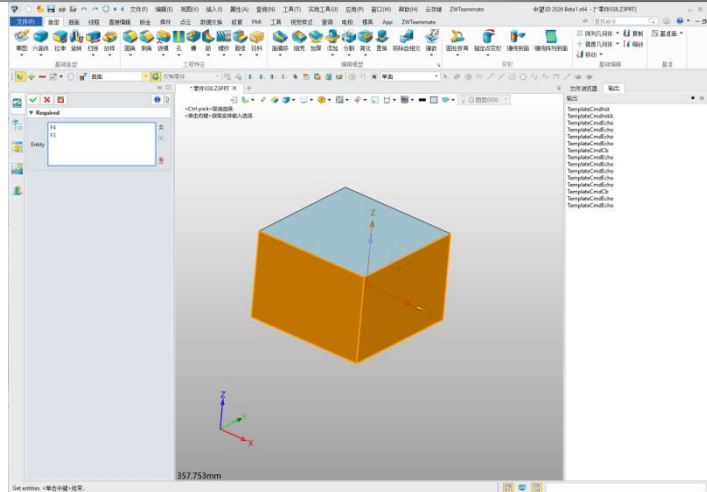
pos > CommandTest > x64 > Debug

Name	Date modified	Type	Size
CommandTest.dll	1/30/2024 3:34 AM	Application exten...	67 KB
CommandTest.exp	1/30/2024 3:34 AM	Exports Library File	2 KB
CommandTest.lib	1/30/2024 3:34 AM	Object File Library	3 KB
CommandTest.pdb	1/30/2024 3:34 AM	Program Debug D...	764 KB
CommandTest.zrc	1/30/2024 3:34 AM	ZRC File	6 KB

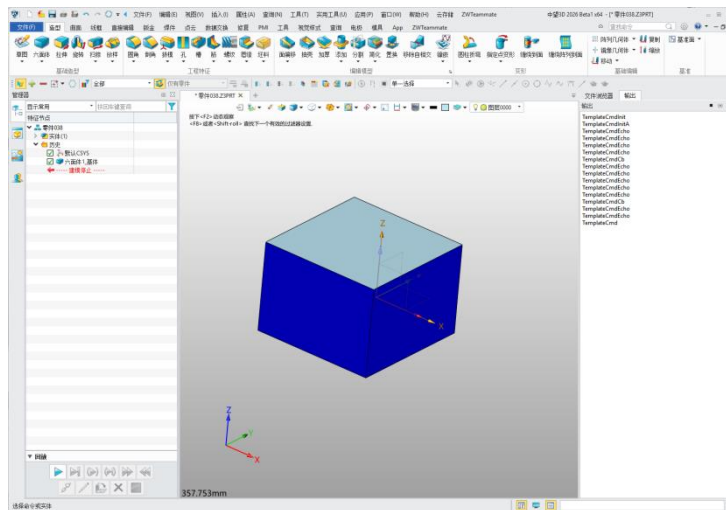
注意：与 DLL 文件不同的是，想要在开启 ZW3D 的状态下更改已加载的 ZRC 文件您需要将 *form* 的 "persistent" 属性设置为 "Temporary"。

j) 效果验证

将 CommandTest.zrc 和 CommandTest.dll 放到 ZW3D 安装目录或用户目录，之后启动 ZW3D（或启动后加载 DLL 也可以）。创建零件环境并绘制一个六面体。在命令中输入 "!TemplateCmd"，您会看到图中的命令对话框。您可以选择一定的面对象。

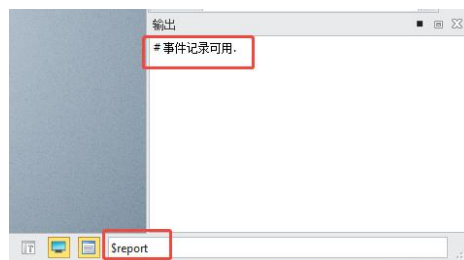


k) 点击确定后您会发现选中的面颜色变为蓝色。



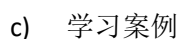
4. ZW3D 命令对话框

a) 在 ZW3D 命令行里运行\$report。命令运行成功后，您会得到以下信息。



b) 运行 ZW3D 原生功能

运行您想要知道的命令，比如创建六面体对应的功能。在信息输出栏您会得到如下信息，[vxSend,"!FtAllBox"]，表示 ZW3D 运行命令!FtAllBox 创建六面体。

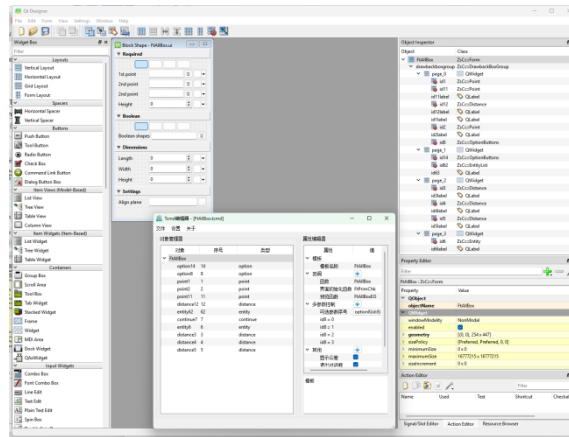


zw3d-command-dialog_2026 > forms 在 forms 中搜索

名称	修改日期	类型	大小
ACTCui	2024/7/18 11:26	Qt UI File	19 KB
ActiveObjForm.ui	2024/7/18 11:26	Qt UI File	4 KB
AdvMilFlowParam.ui	2024/7/18 11:26	Qt UI File	6 KB
AdvMilLeadParam.ui	2024/7/18 11:26	Qt UI File	13 KB
AnimAddAppCtrlForm.ui	2024/7/18 11:26	Qt UI File	7 KB
AnimAddMotor.ui	2024/7/18 11:26	Qt UI File	14 KB
AnimAddViewCtrlForm.ui	2024/7/18 11:26	Qt UI File	6 KB
AnimAlignCmd.ui	2024/7/18 11:26	Qt UI File	8 KB
AnimBandParam.ui	2024/7/18 11:26	Qt UI File	9 KB
AnimBandAnalTestTool.ui	2024/7/18 11:26	Qt UI File	5 KB
AnimCompPos.ui	2024/7/18 11:26	Qt UI File	4 KB
AnimEditCtrlTime.ui	2024/7/18 11:26	Qt UI File	4 KB
AnimImportList.ui	2024/7/18 11:26	Qt UI File	6 KB
AnimInferCtrl.ui	2024/7/18 11:26	Qt UI File	6 KB
AnimInferList.ui	2024/7/18 11:26	Qt UI File	5 KB
AnimModAlignParam.ui	2024/7/18 11:26	Qt UI File	9 KB
AnimModMotParam.ui	2024/7/18 11:26	Qt UI File	10 KB
AnimMotorName.ui	2024/7/18 11:26	Qt UI File	5 KB

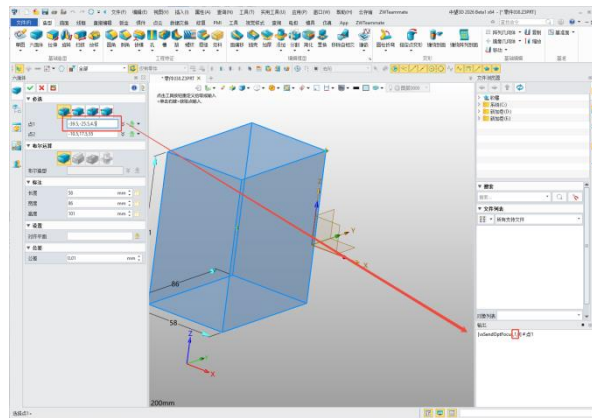
d) UI 与命令对话框

前文介绍了 ZW3D 的 UI 及 Tcmd 文件，您可以借助 ZW3D UI Designer 及 Tcmd 编辑器的方式来生成所需资源，相关细则请参考资源文件及 Tcmd 编辑器帮助文档。



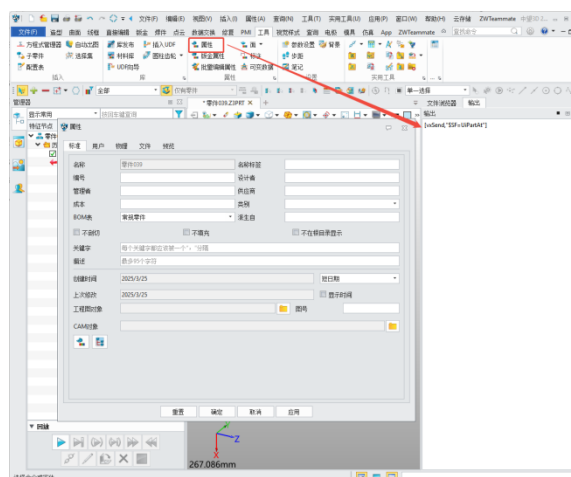
e) 控件与命令

除了直接查看 UI 文件外，通过在界面上设置焦点的方式同样可以获取控件的 ID 等信息，反过来您也可以利用该信息去直接为命令传参。



f) 更多命令

可以发送"\$SF=表单名"去弹出指定的 UI 界面。



5. 通过命令的方式创建六面体

在知道函数名称和控件 ID 的情况下，您可以通过 `cvxDataSet` 等命令的方式直接调



用相关功能，并且利用接口传参的方式直接完成建模。以上文创建六面体的功能为例，尝试调用 ZW3D 命令的方式创建实体，当然您也使用我们提供的 API 函数 `cvxPartBox` 来创建六面体，您可以自主体验两种创建方式的异同。

a) 利用二次开发向导模板创建新项目

参考第二章内容，使用二次开发向导模板创建工程 `CommandTest` 并向头文件 `CommandTestPr.h` 中添加 `RegisterMyBoxCommand` 和 `UnloadMyBoxCommand` 的函数声明。由于命令控制用到了其他接口，需要添加相应接口的头文件。

```

1  #pragma once
2  #include "zwapi_cmd_paramdefine_param.h"
3  #include "zwapi_cmd_paramdefine_tpl.h"
4  #include "zwapi_cmd.h"
5  #include "zwapi_memory.h"
6  #include "zwapi_message.h"
7
8  /*Function declaration*/
9  int RegisterCustomCommand(void);
10 int UnloadCustomCommand(void);
11 int RegisterTemplateCommand(void);
12 int UnloadTemplateCommand(void);
13 int RegisterFormCommand(void);
14 int UnloadFormCommand(void);
15 int RegisterMyBoxCommand(void);
16 int UnloadMyBoxCommand(void);
17

```

b) 添加到初始化及退出函数中

在 `main.cpp` 的初始化函数 `CommandTestInit` 和退出函数 `CommandTestExit` 中分别添加 `RegisterMyBoxCommand` 和 `UnloadMyBoxCommand` 使得函数能够被 ZW3D 正确地加载和卸载。

```

1  #include "..\inc\CommandTestPr.h"
2
3  //Dynamic library entry function, called when the dll is loaded
4  //The function name must be dll name + "Init"
5  int CommandTestInit()
6  {
7      RegisterCustomCommand();
8      RegisterTemplateCommand();
9      RegisterFormCommand();
10     RegisterMyBoxCommand();
11     return 0;
12 }
13
14 //Dynamic library entry function, called when the dll is unloaded
15 //The function name must be dll name + "Exit"
16 int CommandTestExit()
17 {
18     UnloadCustomCommand();
19     UnloadTemplateCommand();
20     UnloadFormCommand();
21     UnloadMyBoxCommand();
22     return 0;
23 }

```

c) 在 `src` 文件夹下创建 `MyBox.cpp`，将下列代码复制粘贴到其中

```

#include "..\inc\CommandTestPr.h"

int myBox();

int RegisterMyBoxCommand()
{

```



```

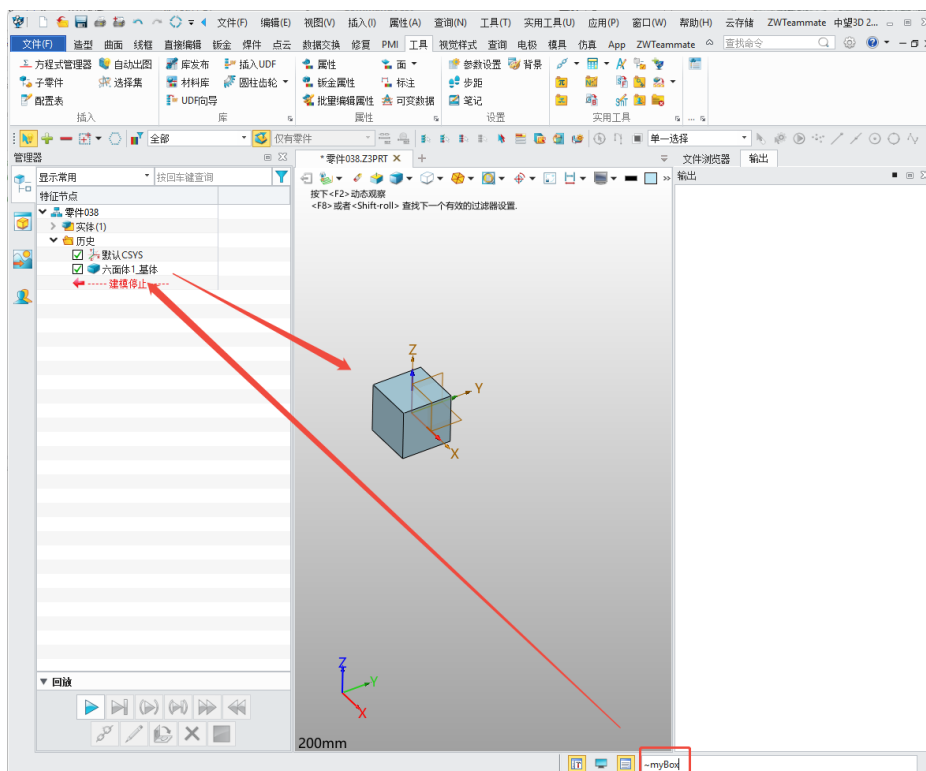
        cvxCmdFunc("myBox", (void*)myBox, VX_CODE_GENERAL);
        return 0;
    }
    int UnloadMyBoxCommand()
    {
        cvxCmdFuncUnload("myBox");
        return 0;
    }
    int myBox()
    {
        int idData = 0;
        svxData tempData;
        cvxDataInit("FtAllBox", &idData);
        // Set the option to choose the type
        cvxDataZero(&tempData);
        tempData.isNumber = 1;
        tempData.NumType = VX_NUM;
        tempData.Num = 1;
        cvxDataSet(idData, 8, &tempData);
        // Set the first point
        cvxDataZero(&tempData);
        tempData.isPoint = 1;
        tempData.PntType = VX_PNT3_ABS;
        tempData.Pnt.x = tempData.Pnt.y = tempData.Pnt.z = 0;
        cvxDataSet(idData, 1, &tempData);
        // Set the second point
        cvxDataZero(&tempData);
        tempData.isPoint = 1;
        tempData.PntType = VX_PNT3_ABS;
        tempData.Pnt.x = tempData.Pnt.y = tempData.Pnt.z = 15;
        cvxDataSet(idData, 2, &tempData);
        // Run this ZW3D command
        cvxCmdExec(idData);
        return 0;
    }
}

```

注意：使用命令调用的方式创建对象时，您需要关注参数原本的默认值，当参数原本的默认值不为0时，您需要进行相应的设置，否则结果可能和您的预想存在差异。您可以通过观察UI和Tcmd文件来了解一个命令的相关参数和设置区域的id。另外您可以对照功能接口与命令之间的异同来自行决定您的实现方式。

d) 生成项目，在ZW3D中加载DLL

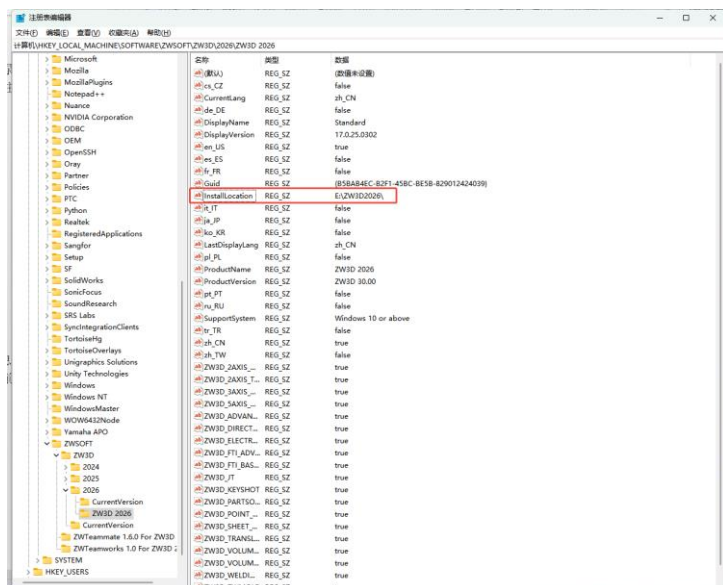
新建零件后，在命令行中运行"~myBox"。这个命令会在建模区新建一个六面体。



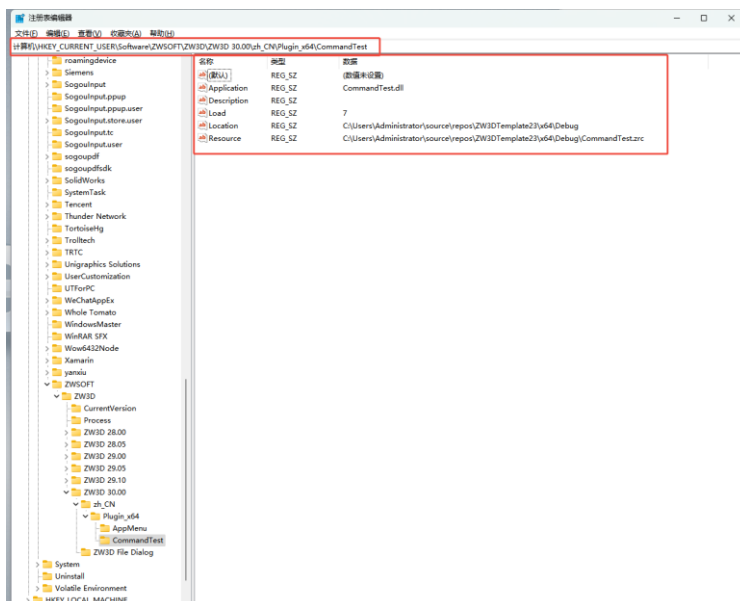
第六章 常见问题解答

1. ZW3D 注册表信息怎么查询？

ZW3D 的注册表信息保存在注册表中，打开注册表管理器，您可以在 "HKEY_LOCAL_MACHINE\SOFTWARE\ZWSOFT\ZW3D" 中找到 ZW3D 的版本信息。根目录中的 "CurrentVersion" 将记住上次运行的版本。特殊版本目录里的 "CurrentVersion" 将记住上次运行的语言。您可以在每个语言目录下获取详细信息，比如安装路径。



注册表管理器中的"HKEY_CURRENT_USER\Software\ZWSOFT\ZW3D"包含资源信息。参考第二章 5 中设置插件配置表，您可以在对应版本的"Plugin_x64"中查看和管理当前的插件配置信息，并完成插件的自动装卸载。





2. DLL 和 ZRC 等相关资源应该放到哪里来让 ZW3D 启动时能自动加载它们？

请将 DLL 和 ZRC 文件放到用户目录或者安装目录的"apilibs"文件夹下，当没有这个文件夹时请自行创建添加。我们更推荐您将文件放在安装目录的"apilibs"文件夹下，避免清理用户目录时造成不必要的资源丢失。

3. 当执行命令时 ZW3D 报错"警告：搜寻 xxx：无该类型 失败"或其他信息时？

可能由以下的原因导致：

- 初始化函数的名称必须与 DLL 文件匹配，即以 DLL 文件的名称加 Init 组成。例如当您的 DLL 名称为"DrawBox"时，您的初始化函数命名需要是"DrawBoxInit"。
- DLL 缺少函数导出，您需要在模块定义文件（.def 文件）中声明 DLL 导出函数。
- 命令缺少注册，您需要使用 `cvxCmdFunc()` 来注册命令。
- 当命令存在与之匹配的 Tcmd 文件时，此时该命令类型为模板命令，您需要用感叹号（!）来唤起它，而不是使用（~）。
- 当您使用感叹号（!）前缀也不能唤起命令时，可能是 ZRC 文件没有和 DLL 处于相同目录下，ZRC 文件可能未被添加到"apilibs"文件夹下，或者您忘记重新编译资源文件。当您需要重新加载 ZRC 文件时，请重新启动 ZW3D 来保证资源的完整性。

4. 什么是命令的注册名称？

命令的注册名称是用于注册函数的关键字，比如

```
ZwCommandFunctionLoad("CreateHole", (void*)ModelAddHole,  
ZW_LICENSE_CODE_GENERAL);
```

"CreateHole" 就是命令的注册名称。

"ModelAddHole" 是命令的实现函数指针。

"ZW_LICENSE_CODE_GENERAL"是指示命令的授权。

您可以使用"~"+命令的注册名称的方式（~CreateHole）来唤起您定义的函数。当您使用模板命令时需要主要保持命令的注册名称与 Tcmd 文件的模板名称以及 UI 的文件名称一致，为了更好地保证您的使用稳定性，我们建议您在模板命令时，命令的注册名称采用与 Tcmd 文件名（及模板名）和 UI 文件名相同的名称。

5. 命令的注册名称长度限制是多少？

不超过 128 ASCII 字符的长度，当超过时文本将被截断，因此请注意您的注册名称长度。

6. 什么是 Custom 特征，它与一般特征有什么区别？

在 ZW3D 中当您想要将两个及以上的功能组合成一个特征时，您可以借助 Custom 特征的方式来创建对象。您需要在 Tcmd 文件中设置执行函数为 CustomOp，并添加 custom 属性为对应客制函数。例如"FilletBox"函数需要设置 custom 为.c（不可炸开，删除子特征）或.cc（不可炸开，不删除子特征）。

```
<property name="function">CustomOp</property>  
<property name="custom">FilletBox.c</property>
```

需要注意的是在使用 Custom 特征的情况下，其内部的子特征在做布尔类型的操作时，不支持自动登记。即您需要明确指定布尔操作的基体和工具体，否则会导致特征创建失败。

7. 为什么有的 API 函数会生成历史记录而有的不会？

如果 API 函数不修改任何实体，仅做一些查询或创建文件的工作则不会生成



历史记录。原则上一个操作能通过与 ZW3D 的交互生成历史时，那么执行相同的 API 函数会生成相同的历史。

8. 为什么有时候字体和文本会设置失败或者出现乱码？

ZW3D 内部默认使用 UTF-8 的编码方案，当您设置的字体或文本失败或出现乱码时，可以尝试使用 UTF-8 的方式进行输入



第七章 ZW3D 特殊符号转义字符映射表

注意：ZW3D 符号列双引号内的文本对应 ZW3D 特殊符号。'31' 使用 ACSII 代码，在键入时需要特殊处理。'31'的十六进制 ASCII 代码是 0x1F，'31'的十六进制 ASCII 代码是 0x7F。

映射关系		符号	Maps		符号	Maps		符号
字符串	Unicode		字符串	Unicode		字符串	Unicode	
[VxX]#	\U+E012		[VxY]#	\U+E06E				
[VxX]\$	\U+E00D		[VxY]\$	\U+E06F				
[VxX]%	\U+E00C		[VxY]%	\U+E070				
[VxX]&	\U+E056		[VxY]&	\U+E071				
[VxX]'	\U+E02D		[VxY]'	\U+E072				
[VxX](	\U+E02B		[VxY](	\U+E073				
[VxX])	\U+E01A		[VxY])	\U+E074				
[VxX]*	\U+E00B		[VxY]*	\U+E075				
[VxX]+	\U+E005		[VxY]+	\U+E076				
[VxX],	\U+E04E		[VxY],	\U+E077				
[VxX]-	\U+E04C		[VxY]-	\U+E078				
[VxX].	\U+E04A		[VxY].	\U+E079				
[VxX]/	\U+E01C		[VxY]/	\U+E07A				
[VxX]0	\U+E01B		[VxY]0	\U+E07B				
[VxX]1	\U+E02C		[VxY]1	\U+E07C				



[VxX]2	\U+E019		[VxY]2	\U+E07D				
[VxX]3	\U+E055		[VxY]3	\U+E07E				
[VxX]4	\U+E054							
[VxX]5	\U+E022		[VxY]5	\U+E080				
[VxX]6	\U+E02A		[VxY]6	\U+E081				
[VxX]7	\U+E029		[VxY]7	\U+E082				
[VxX]8			[VxY]8	\U+E083				
[VxX]9			[VxY]9	\U+E084				
[VxX]:	\U+E05C		[VxY]:	\U+E085				
[VxX];	\U+E05D		[VxY];	\U+E086				
[VxX]<	\U+E059		[VxY]<	\U+E087				
[VxX]=	\U+E058		[VxY]=	\U+E088				
[VxX]>	\U+E05A		[VxY]>	\U+E089				
[VxX]?	\U+E05B		[VxY]?	\U+E08A				
[VxX]@			[VxY]@	\U+E08B				
[VxX]A			[VxY]A	\U+E08C		[VxWD]A		
[VxX]B			[VxY]B	\U+E08D		[VxWD]B		
[VxX]C			[VxY]C	\U+E08E		[VxWD]C		
[VxX]D			[VxY]D	\U+E08F		[VxWD]D		
[VxX]E			[VxY]E	\U+E090		[VxWD]E		



[VxX]F			[VxY]F	\U+E091		[VxWD]F		
[VxX]G			[VxY]G	\U+E092		[VxWD]G		
[VxX]H						[VxWD]H		
[VxX]I						[VxWD]I		
[VxX]J						[VxWD]J		
[VxX]K						[VxWD]K		
[VxX]L						[VxWD]L		
[VxX]M						[VxWD]M		
[VxX]N						[VxWD]N		
[VxX]O						[VxWD]O		
[VxX]P						[VxWD]P		
[VxX]Q						[VxWD]Q		
[VxX]R						[VxWD]R		
[VxX]S						[VxWD]S		
[VxX]T						[VxWD]T		
[VxX]U						[VxWD]U		
[VxX]V						[VxWD]V		
[VxX]W						[VxWD]W		
[VxX]X	\U+E065					[VxWD]X		
[VxX]Y	\U+E04F					[VxWD]Y		



[VxX]Z	\U+E051					[VxWD]Z	
[VxX][						[VxWD][	
[VxX]\	\U+E053					[VxWD]\	
[VxX]]						[VxWD]]	
[VxX]^	\U+E027					[VxWD]^	
[VxX]_	\U+E028					[VxWD]_	
[VxX]`	\U+E00E					[VxWD]`	
[VxX]a	\U+E041					[VxWD]a	
[VxX]b	\U+E049					[VxWD]b	
[VxX]c	\U+E042					[VxWD]c	
[VxX]d	\U+E03C					[VxWD]d	
[VxX]e	\U+E033					[VxWD]e	
[VxX]f	\U+E040					[VxWD]f	
[VxX]g	\U+E03F					[VxWD]g	
[VxX]h	\U+E034					[VxWD]h	
[VxX]i	\U+E03E					[VxWD]i	
[VxX]j	\U+E03D					[VxWD]j	
[VxX]k	\U+E011						



[VxX]l	\U+E03A							
[VxX]m	\U+E03B					[VxWD]m		
[VxX]n	\U+E039							
[VxX]o	\U+E00F					[VxWD]o		
[VxX]p	\U+E010					[VxWD]p	\U+E06C	
[VxX]q	\U+E038							
[VxX]r	\U+E037							
[VxX]s	\U+E036							
[VxX]t	\U+E025							
[VxX]u	\U+E026							
[VxX]v	\U+E024							
[VxX]w	\U+E023							
[VxX]x	\U+FF5C							
[VxX]y	\U+E031							
[VxX]z	\U+E030							
[VxX]{	\U+E032							
[VxX]	\U+E02E							
[VxX}]	\U+E02F							
[VxX]~	\U+E020							



[VxX]	\U+E050							
[VxX]!	\U+E052		[VxY]!	\U+E07F		[VxWD]!	\U+2190	
[VxX]	\U+E053		[VxY]	\U+E06D				
[VxX]'127'	\U+E06B							
[VxX]'31'	\U+E04D							